UNIVERSITY POLITEHNICA OF BUCHAREST FACULTY OF AUTOMATIC CONTROL AND COMPUTERS COMPUTER SCIENCE DEPARTMENT





MASTER'S THESIS

ABUSIVE LANGUAGE DETECTION ON SOCIAL MEDIA PLATFORMS

Ahmed Raza

Coordonator științific:
Prof. dr. ing. Traian Eugen Rebedea

BUCUREŞTI

2026

Contents

Abstract **3**
Acknowledgments **4**
1. Introduction **5**
1.1 What is Abusive Language Detection? 5
1.2 Why Mix Symbolic and Statistical Learning? 5
1.2.1 Traditional approaches rely either on: 5
1.3 Why Focus on Identity-Hate? 5
1.4 Why Use NLP Instead of Basic Filters? 6
1.5 What's the Goal of This Research? 6
2. Background and Foundations **6**
2.1 Dataset Innovation 7
2.2 GPU-Accelerated Experimentation 7
2.3 Rigorous Benchmarking 8
2.4 Sequential Statistical Learning Techniques 8
3. NLP Approaches to Abusive Language Detection **9**
3.1 Hybrid Model Architecture 9
3.2 Dataset Fusion and Balancing 10
3.3 Contextual Analysis 10
3.4 Performance-Aware Training 10
3.5 Core Algorithms 11
4. Review of Previous Research **11**
4.1 Research Review Summary 11
4.2 Gap Identification 12
5 Dataset Implementations **12**
5.1 Data Acquisition 12
5.2 Dataset Structure and Integration 13
5.3 Data Preparation **14**
5.3.1 Dataset Sources 14
DS1: Kaggle Abusive Language Dataset 15
DS2: Twitter Hate Speech Dataset 15
DS3: Processed Lyrics Dataset 15
DS4: Identity Hate Dataset 15
5.3.2 Data Loading and Merging 16
DS1 Processing 16
DS2 Processing 16
DS3 and DS4 Integration 16
Final Combined Dataset 16
5.3.3 Data Splitting 16
5.3.4 Dataset Characteristics 17

Multi-Label Nature....17
Class Imbalance.... 17
Textual Diversity....17
5.4 Annotation and Labeling.... 17
5.5 Challenges and Limitations....18
5.6 Comment Length and Data Corelation.... 19
5.7 Dataset Expansion for Research 4....21
5.8 Data Visualization and Comparative Analysis.... 23
6. Model Architectures.... 25
6.1 LSTM Model Architecture.... 25
6.1.1Model Architecture....25
6.1.2 Optimization and Regularization....25
6.2 BERT Model Architecture.... 25
6.2.1 Data Preparation....26
6.2.2 Model Architecture....26
6.2.3 Training and Evaluation.... 26
7. Models Training and Testing.... 26
7.1 LSTM Model Training and Results....26
7.1.1 Data Loading and Preprocessing....27
7.1.2 Data Splitting and Tokenization....27
7.1.3 Model Architecture and Training....27
7.1.4 Evaluation and Visualization....27
7.2 BERT Basic Model Training and Results....30
7.1.1 Data Loading and Preprocessing....30
7.1.2 Data Splitting and Tokenization....30
7.1.3 Model Architecture and Training....30
7.1.4 Evaluation and Visualization....30
7.3 RoBERTa Model Training and Results.... 32
7.4 DeBERTaV3 Model Training and Results....34
7.5 ToxicBERT Model Training and Results.... 36
7.6 ToxicBERT Model Training and Results - Phase 4 (Extended Dataset)....38
8. Future Directions....40
8.1 Dataset Expansion and model Enhancement....40
8.2 Training Optimization and Deployment....40
9. Conclusion.... 41
10. Bibliography....42

Abstract

The proliferation of abusive language on social media—particularly identity-based hate speech—demands advanced detection systems that are both accurate and context-aware. While existing methods leverage deep learning models like BERT and RoBERTa, challenges persist in handling imbalanced datasets and subtle forms of targeted toxicity. This research addresses these gaps by (1) combining and rebalancing two benchmark datasets to improve representation of identity-hate content and (2) proposing a hybrid architecture that integrates Toxic-BERT with domain-specific adaptations (e.g., DeBERTa) for enhanced contextual understanding.

Our primary objectives are to (i) optimize classification performance for identity-hate speech by mitigating dataset bias through strategic sampling and augmentation; (ii) design a comparative framework to evaluate hybrid models (BERT/RoBERTa/DeBERTa) against baseline toxic-language detectors; and (iii) quantify trade-offs between precision and recall in real-world scenarios. The proposed solution targets social media platforms where rapid, interpretable moderation is critical, with validation on curated datasets emphasizing racial, gender, and religious harassment. By unifying fragmented datasets and refining transformer-based models, this work aims to advance the robustness of abusive language detection while providing actionable insights for ethical AI deployment.

Key challenges identified include model interpretability, cross-platform generalizability, and the ethical implications of automated moderation. The study further proposes future directions, such as integrating multimodal data (text + metadata) and leveraging user feedback loops to refine detection accuracy. By systematically comparing proposed solutions against existing baselines, this work aims to contribute actionable insights for both researchers and social media platforms in combating online abuse effectively.

In the last phase of our work, we introduced 115K examples of the lyrics dataset and annotated them on the basis of the previously defined rules and trained toxic_bert with pretty impressive results that we will discuss later on in the upcoming results section.

Acknowledgments

Writing this dissertation would have been an impossible task if it were not for the support, guidance, and encouragement I received from those people to whom I am thankful.

First of all, I owe a great deal of gratitude to my thesis advisor, Dr. Traian Eugen Rebedea, who was very professional in supervising me during my work on this study. His wise tips were of great help in carrying out this research.

I would also like to express my gratitude towards the teaching staff of the Computer Science Department, Faculty of Automatic Control and Computers, University Politehnica of Bucharest, for providing me with excellent knowledge and an inspiring atmosphere for my academic study.

I would also like to show my appreciation for my colleagues and friends who provided me with fruitful debates, faith, and relaxation bursts to make this scholarly adventure easy and interesting.

I would like to express my deepest gratitude to my family for their understanding, tolerance, and support throughout this endeavor.

1. Introduction

1.1 What is Abusive Language Detection?

Abusive language detection is all about teaching computers to spot harmful content online—things like hate speech, harassment, or targeted attacks. Social media platforms deal with millions of posts every day, and manually filtering out toxic content is nearly impossible. That's where automated systems come in. But it's not as simple as just flagging curse words or slurs. Real-world abusive language is sneaky—it hides behind sarcasm, coded language, or even seemingly harmless phrases that carry hateful meaning in certain contexts. Our work zeroes in on identity-based hate speech—content that attacks people based on race, religion, gender, or other personal traits. This type of abuse is especially damaging because it fuels real-world discrimination and violence, making accurate detection a top priority.

1.2 Why Mix Symbolic and Statistical Learning?

Most detection systems today take one of two paths: either they rely on strict rules (like a list of banned words), or they use machine learning to find patterns in data. Rule-based systems are easy to understand—if someone posts a known slur, the system catches it. But they fail when people get creative with spelling or slang. On the other hand, pure machine learning models, like BERT, are great at picking up hidden patterns but can miss obvious hate speech if it wasn't in their training data.
Our approach combines the best of both worlds. We use symbolic methods (like keyword filters and hate speech lexicons) to catch clear-cut cases, while statistical NLP models (like fine-tuned BERT) handle the tricky, context-dependent stuff. For example, the word "dog" might be harmless in most posts but could be a racial slur in certain contexts. A hybrid system can spot the difference where others might fail.

1.2.1 Traditional approaches rely either on:

- **Symbolic rules** (e.g., lexicons of hate terms): Interpretable but fail to generalize.
- **Statistical models** (e.g., TF-IDF, SVM): Capture patterns but ignore linguistic nuance.
  Our hybrid approach bridges this gap:
- **Symbolic methods** flag known hate markers (e.g., "k*ll all [group]").
- **Statistical NLP** (e.g., BERT variants) infers context (e.g., "They're killing it" vs. violent intent). This synergy improves robustness against adversarial evasion (e.g., misspellings, sarcasm).

1.3 Why Focus on Identity-Hate?

Hate speech isn't just about offensive words—it's about who those words target. Identity-based attacks (like racism, sexism, or religious hate) spread real harm, pushing people out of online spaces and sometimes inciting violence. The problem? Most existing datasets and tools aren't built to handle the nuances of identity-hate. They might do okay with blatant slurs but struggle with subtler forms, like dog whistles or backhanded compliments.

To fix this, we're combining and rebalancing two popular hate speech datasets (HateXplain and Civil Comments) to better represent identity-based abuse. This way, our models learn to recognize not just obvious hate but also the sneaky, indirect kinds that often slip through the cracks.

1.4 Why Use NLP Instead of Basic Filters?

You might wonder—why not just block certain words or phrases? The answer is simple: hate speech evolves faster than any rule list. People change spellings, use coded language, or even reclaim words (like how "queer" can be either a slur or a proud identity label). Static filters can't keep up, but NLP models can adapt.

Our system uses advanced techniques like Toxic-BERT and RoBERTa, fine-tuned specifically for hate speech. Unlike basic keyword blockers, these models understand context. They can tell the difference between a post saying "I'm so gay" (a positive statement) and one saying "That's so gay" (used as an insult). This flexibility is key for accurate moderation.

1.5 What's the Goal of This Research?

The big picture? We want to build a detection system that's both accurate and fair—one that catches real hate speech without unfairly silencing marginalized voices. To get there, we're tackling three main challenges:

- Better Data: Merging and balancing datasets to reduce bias and improve coverage of identity-hate. Introducing new datasets.
- Smarter Models: Designing hybrid systems that combine symbolic rules with deep learning for the best of both worlds.
- Real-World Testing: Evaluating performance not just on clean lab data but on messy, real social media posts.

2. Background and Foundations

Detecting abusive language on social media platforms requires models capable of identifying both explicit hate speech, such as racial slurs, and subtle, context-dependent expressions, such as dogwhistles or microaggressions [Waseem and Hovy, 2016]. Existing systems often face challenges, including dataset

biases that limit generalizability, the computational complexity of advanced natural language processing (NLP) models, and the need for interpretable detection methods [Nobata et al., 2016]. Our research addresses these issues through a hybrid approach that integrates symbolic rule-based filters with transformer-based models, such as BERT [Devlin et al., 2018] and RoBERTa [Liu et al., 2019]. To establish a robust foundation, we prioritize five key areas: dataset innovation, GPU-accelerated experimentation, rigorous benchmarking, sequential statistical learning techniques, and advanced NLP techniques using transformer models. These pillars ensure our solutions are technically sound, scalable, and effective for real-world content moderation, with Kaggle's collaborative environment enabling transparent result validation.

2.1 Dataset Innovation

Robust abusive language detection relies on diverse, representative training data, yet many datasets, such as HateXplain [Mathew et al., 2021] and Civil Comments [Koh et al., 2019], exhibit class imbalances that favor overt slurs over subtle or identity-specific hate. Our approach innovates by merging these corpora to form a comprehensive training set that captures a broad range of abusive language patterns. We address imbalances using the Synthetic Minority Oversampling Technique (SMOTE) [Chawla et al., 2002] to augment underrepresented categories, such as antisemitic or gender-based hate, and employ stratified sampling to mitigate implicit bias, ensuring equitable representation across demographic groups. This method improves model performance on nuanced content, a critical requirement for real-world applications [Davidson et al., 2017].

Additionally, we preprocess datasets to standardize text formats, remove irrelevant metadata, and preserve contextual cues essential for accurate classification. By integrating multiple sources and balancing their distributions, we overcome limitations in prior work, where models struggled with covert hate due to skewed data [Swamy et al., 2019]. This pillar lays the groundwork for training models that generalize effectively across diverse social media platforms, enhancing detection accuracy and fairness.

2.2 GPU-Accelerated Experimentation

Training sophisticated NLP models, such as DeBERTa [He et al., 2020] and ToxicBERT [Caselli et al., 2021], requires substantial computational resources to process large datasets and optimize complex architectures. We leverage Kaggle Kernels, a GPU-accelerated platform, to facilitate rapid experimentation with our hybrid model, which combines symbolic rule-based filters with transformer fine-tuning [Jurafsky and Martin, 2022]. This infrastructure supports efficient hyperparameter tuning—adjusting learning rates, batch sizes, and model configurations—while significantly reducing training times compared to CPU-based systems. Our iterative process refines the synergy between symbolic rules for explicit hate detection and transformers for contextual analysis, ensuring both precision and scalability.

Kaggle's collaborative ecosystem provides access to diverse datasets and pre-trained models, enabling reproducible benchmarks that validate our approach. This builds on prior GPU-accelerated NLP research [Liu et al., 2019], but uniquely applies it to abusive language detection, addressing challenges like overfitting and computational cost [Bengio, 2012]. By optimizing model designs for real-world scenarios, this pillar ensures our solution is practical for deployment on resource-constrained platforms.

2.3 Rigorous Benchmarking

To validate our hybrid model's effectiveness, we conduct rigorous benchmarking by comparing performance metrics—F1 scores, precision-recall trade-offs, and computational costs—across models, including logistic regression baselines and transformers like RoBERTa and ToxicBERT [Caselli et al., 2021]. We evaluate models on curated datasets, such as Jigsaw's Toxicity dataset [2], to ensure generalizability across platforms with varied linguistic norms. This process balances precision, to avoid mislabeling benign content, and recall, to capture subtle identity-based hate, where under-detection has significant consequences [Mathew et al., 2021].

Our benchmarking also assesses computational efficiency, critical for real-time moderation, by logging model decisions and performance metrics for transparency [Rogers et al., 2020]. Unlike prior studies that often lacked comprehensive cross-model comparisons [Swamy et al., 2019], our approach provides detailed insights into system performance, enabling developers to deploy robust, fair, and efficient detection systems. This pillar reinforces the reliability of our contributions to abusive language detection research.

2.4 Sequential Statistical Learning Techniques

Our research innovates by incorporating sequential statistical learning techniques to enhance the detection of abusive language, particularly in dynamic social media contexts where hate speech evolves rapidly. We build on traditional statistical models, such as logistic regression and support vector machines, which identify predictive features like specific slurs [Davidson et al., 2017], and extend them with sequential approaches, such as recurrent neural networks (RNNs) and long short-term memory (LSTM) models [Hochreiter and Schmidhuber, 1997]. These models capture temporal and syntactic dependencies in text, enabling the detection of hate speech that spans multiple words or sentences, such as veiled threats or rhetorical biases. For example, an LSTM can analyze the sequence "Why do [group] always cause trouble?" to detect implied prejudice, even without explicit slurs.

To address the limitations of RNNs, such as vanishing gradients, we integrate transformer-based models, which leverage attention mechanisms to process sequences more effectively [Vaswani et al., 2017]. By combining sequential statistical learning with our hybrid architecture, we ensure the model adapts to evolving linguistic patterns, improving robustness against adversarial inputs like misspellings or coded language. This approach distinguishes our work from earlier studies that relied solely on static feature extraction [Nobata et al., 2016], offering a flexible framework for real-world deployment.

3. NLP Approaches to Abusive Language Detection

Modern abusive language detection requires models capable of identifying both explicit hate speech, such as racial slurs, and subtle, context-dependent expressions, such as coded language or microaggressions. Unlike traditional keyword-based systems, which rely on static lexicons and struggle with evolving linguistic patterns [Nobata et al., 2016], our approach integrates symbolic rule-based filtering with transformer-based models to achieve robust performance. Symbolic systems employ curated lexicons and regular expression (regex) patterns to detect overt hate speech, such as violent threats, while statistical models like BERT [Devlin et al., 2018] and RoBERTa [Liu et al., 2019] analyze syntactic and semantic relationships to uncover implicit bias. This hybrid architecture combines the interpretability of rule-based methods with the contextual understanding of deep learning, ensuring comprehensive detection without relying solely on brittle keyword lists or opaque neural networks.

A key challenge is managing the high dimensionality of text data when merging multiple datasets, which can complicate model training. To address this, we adopt a modular approach: symbolic rules pre-filter clear cases of hate speech, reducing the computational load on transformer models. These transformers are then fine-tuned on specific hate speech categories, such as identity-based harassment, to enhance their ability to detect nuanced or targeted abuse. This pipeline optimizes both efficiency and accuracy, enabling scalable deployment on social media platforms.

3.1 Hybrid Model Architecture

Our system integrates symbolic and statistical techniques to maximize precision and recall in abusive language detection. Symbolic methods, such as hate word lexicons and syntactic pattern matching, are computationally efficient and interpretable, excelling at identifying explicit slurs or threats [Davidson et al., 2017]. However, they struggle with misspellings, slang, or context-dependent expressions. In contrast, transformer models like BERT [Devlin et al., 2018] and DeBERTa [He et al., 2020] capture nuanced context but are computationally intensive and prone to overfitting on imbalanced datasets. By combining these approaches, we create a pipeline where symbolic filters handle straightforward cases, allowing transformers to focus on ambiguous or context-sensitive content.

For example, a post stating “all [group] are criminals” is flagged by the symbolic layer for its stereotypical structure, while the transformer model assesses whether the tone is hateful or critical based on contextual cues. This dual-layer approach mitigates dataset biases by leveraging manually curated rules to compensate for underrepresented hate categories in training data, ensuring broader coverage [Mathew et al., 2021]. The pipeline’s design draws inspiration from ensemble methods in NLP, where complementary models improve overall performance [Dietterich, 2000], making it suitable for real-time moderation on platforms with diverse linguistic patterns.

3.2 Dataset Fusion and Balancing

Robust abusive language detection requires diverse, representative training data. However, most hate speech datasets, such as HateXplain [Mathew et al., 2021] and Civil Comments [Koh et al., 2019], suffer from class imbalances, overemphasizing overt slurs while underrepresenting subtle or identity-specific hate. To address this, we merge these corpora to create a comprehensive training set. HateXplain provides detailed annotations for identity-based hate, while Civil Comments encompasses a wider range of toxicity types, enabling our model to capture varied abusive language patterns encountered in real-world scenarios.

To mitigate class imbalance, we apply the Synthetic Minority Oversampling Technique (SMOTE) [Chawla et al., 2002] to augment underrepresented categories, such as antisemitic or gender-based hate, ensuring the model prioritizes these critical cases during training. For instance, if antisemitic content is scarce in one dataset, SMOTE generates synthetic samples to balance the distribution, reducing bias toward majority classes. This approach, validated through cross-platform testing on Jigsaw's Toxicity dataset [Jigsaw, 2018], enhances the model's generalizability across diverse demographic targets, resulting in a fairer and more effective detection system.

3.3 Contextual Analysis

Effective hate speech detection hinges on understanding contextual cues, as abusive language often relies on implied or scattered expressions rather than explicit slurs. Transformer models like RoBERTa [Liu et al., 2019] leverage attention mechanisms to analyze word relationships across entire sentences, enabling the detection of rhetorical bias in phrases like "Why do [group] always play the victim?" which may lack overt slurs but signal prejudice through framing [Waseem and Hovy, 2016]. The model's attention heads assign higher weights to contextually significant terms, improving classification accuracy for subtle abuse.

To ensure generalizability, we cross-validate our models on diverse datasets, such as Jigsaw's Toxicity dataset [Jigsaw, 2018], using platforms like Kaggle for robust testing. This process evaluates performance against real-world social media data, where new hate speech patterns emerge rapidly. By training on varied corpora and testing across platforms, we enhance the model's adaptability to dynamic online environments, a critical requirement for effective content moderation.

3.4 Performance-Aware Training

Our training process optimizes for classification performance, balancing precision (minimizing false positives, such as misflagged neutral content) and recall (capturing subtle hate speech) to maximize F1-scores. Using Kaggle's GPU infrastructure, we experiment with hyperparameters—such as batch sizes and learning rates—to identify optimal configurations [Bengio, 2012]. For identity-based hate, where

under-detection poses greater risks, we prioritize higher recall, even at the cost of moderate precision losses, to ensure harmful content is flagged effectively.

We monitor both accuracy and computational efficiency to ensure the hybrid system is practical for real-time deployment. The lightweight symbolic layer maintains baseline performance for explicit cases, while transformers address complex, context-dependent abuse, akin to hierarchical NLP pipelines [Jurafsky and Martin, 2023]. This approach ensures scalability without sacrificing detection quality, making it suitable for large-scale social media platforms.

3.5 Core Algorithms

Our methodology begins with interpretable baselines, such as logistic regression, to establish performance benchmarks, followed by advanced transformer models (BERT [Devlin et al., 2018], RoBERTa [Liu et al., 2019], DeBERTa [He et al., 2020]). Logistic regression identifies predictive features, such as specific slurs, providing a foundation for comparison [Davidson et al., 2017]. Transformers enhance this by incorporating contextual understanding, enabling detection of nuanced hate speech missed by simpler models.

The hybrid system operates as a coordinated pipeline: symbolic rules process straightforward cases, while transformers handle complex, context-sensitive content. All decisions are logged for transparency, facilitating iterative improvements, such as updating regex patterns or retraining transformers on new data. This modular design, inspired by ensemble learning [Dietterich, 2000], ensures both interpretability and robustness, addressing the diverse challenges of abusive language detection.

4. Review of Previous Research

4.1 Research Review Summary

The reviewed studies provide foundational insights into abusive language detection across social media platforms, employing diverse methodologies from traditional machine learning to modern transformer-based approaches. Early work, such as Nobata et al. (2016) and ElSherief et al. (2012), utilized n-grams, lexicons, and logistic regression to detect explicit offensive content on platforms like Yahoo and YouTube, achieving moderate performance (e.g., 0.82 F1) but struggling with subtle hate speech and cross-platform generalizability [1], [2]. More recent studies, like Mossie and Wang (2023) and Bertaglia et al. (2021), advanced the field by applying Word2Vec with RNN-GRU for low-resource languages (92.56% accuracy) and creating legally inspired datasets (ALYT) with BERTweet (0.757 macro F1), respectively, though they faced limitations in handling nuanced linguistic patterns and multilingual contexts [3], [5]. Kandias et al. (2013) explored psychosocial indicators for insider threat detection on YouTube (81% accuracy), but their binary classification lacks alignment with multi-class abusive language taxonomies [4]. Caselli et al. (2021) introduced HateBERT, a BERT variant fine-tuned on Reddit data,

outperforming generic BERT (e.g., 0.809 macro F1 on OffensEval) and demonstrating robustness in detecting abusive content, yet it overlooks recent linguistic trends and microaggressions [6]. Collectively, these studies highlight progress in feature extraction, dataset creation, and model adaptation but reveal persistent challenges in contextual understanding and scalability.

4.2 Gap Identification

Despite their contributions, the reviewed studies exhibit critical gaps that our research addresses through a hybrid approach combining symbolic rules and transformer models. Most studies rely on outdated methods (e.g., static lexicons, Word2Vec) or datasets skewed toward overt toxicity, limiting their ability to detect subtle hate speech, such as sarcasm or dogwhistles, and reducing generalizability across platforms like Twitter or Reddit [1]–[3], [5]. Binary or simplistic taxonomies, as seen in [1], [2], [4], fail to differentiate nuanced categories like identity-based hate or threats, which our six-class framework (toxic, severe toxic, obscene, hate, insult, threat) tackles. The absence of advanced transformers like RoBERTa, DeBERTa, or ToxicBERT in [3], [4], and limited comparisons to state-of-the-art models in [2], [6], hinder performance on complex linguistic phenomena, such as negation or microaggressions. Additionally, small or language-specific datasets (e.g., 14,266 Amharic posts in [3], 900 English comments in [4]) and lack of multilingual support constrain robustness and scalability. Ethical concerns, such as false positives in marginalized communities [2] and annotator bias [1], remain underexplored. Our work bridges these gaps by fusing balanced, diverse datasets (HateXplain, Civil Comments), leveraging advanced transformers, and ensuring fairness-aware design for real-world deployment.

5 Dataset Implementations

This section outlines the collective implementation of the datasets utilized in this research for detecting toxic language across diverse contexts, such as social media interactions, song lyrics, and public discourse. The datasets, namely the Social Media Toxicity Prediction Dataset, Twitter Hate Speech Dataset, Processed Lyrics Dataset, and Identity Hate Dataset, serve as the foundation for training and evaluating natural language processing (NLP) models aimed at identifying harmful content. The implementation process encompasses data acquisition, preprocessing, annotation, and structural organization to ensure compatibility with machine learning workflows. A general approach is adopted to discuss the implementation, focusing on shared methodologies and considerations across all datasets, while addressing challenges such as data quality, bias mitigation, and ethical concerns. This collective perspective aligns with the research objective of developing robust and generalizable models for toxicity detection, applicable to real-world scenarios.

5.1 Data Acquisition

The acquisition of the datasets involved sourcing high-quality, publicly available data from reputable platforms, primarily Kaggle, which hosts a variety of annotated collections for NLP research. The Social

Media Toxicity Prediction Dataset and Processed Lyrics Dataset were obtained from Kaggle's repositories, leveraging their extensive collections of social media comments and song lyrics, respectively. The Twitter Hate Speech Dataset was curated from Twitter's API, focusing on tweets with potential hate speech or offensive content, while the Identity Hate Dataset was manually compiled from diverse online sources to target identity-based prejudice. To ensure relevance to toxicity detection, each dataset was selected based on its annotation quality, scale, and diversity of content. For instance, the Social Media Toxicity Prediction Dataset, with over 159,000 entries, provides a large-scale resource for multi-label classification, while the smaller Identity Hate Dataset, with 1,000 entries, offers a focused perspective on identity-based hate speech. The acquisition process adhered to ethical guidelines, prioritizing datasets with anonymized user information to protect privacy and comply with data usage policies.

5.2 Dataset Structure and Integration

The datasets were uniformly structured as CSV files to facilitate integration into NLP workflows, with each row representing a single entry, such as a comment, tweet, lyric line, or statement, and columns capturing text and labels. The Social Media Toxicity Prediction Dataset comprised 159,571 rows with seven columns, including an id and six probability scores, optimized for multi-label classification evaluation. The Twitter Hate Speech Dataset contained 25,297 rows with six columns—count, hate speech, offensive language, neither, class, and tweet—supporting both text analysis and label-based training. The Processed Lyrics Dataset, with 157,190 rows, and the Identity Hate Dataset, with 1,000 rows, shared an eight-column structure, including id, comment text, and six binary labels, enabling consistent multi-label classification tasks. A diagram illustrating the relative sizes of the datasets is provided below to highlight their scale differences, which influence model training and generalizability.

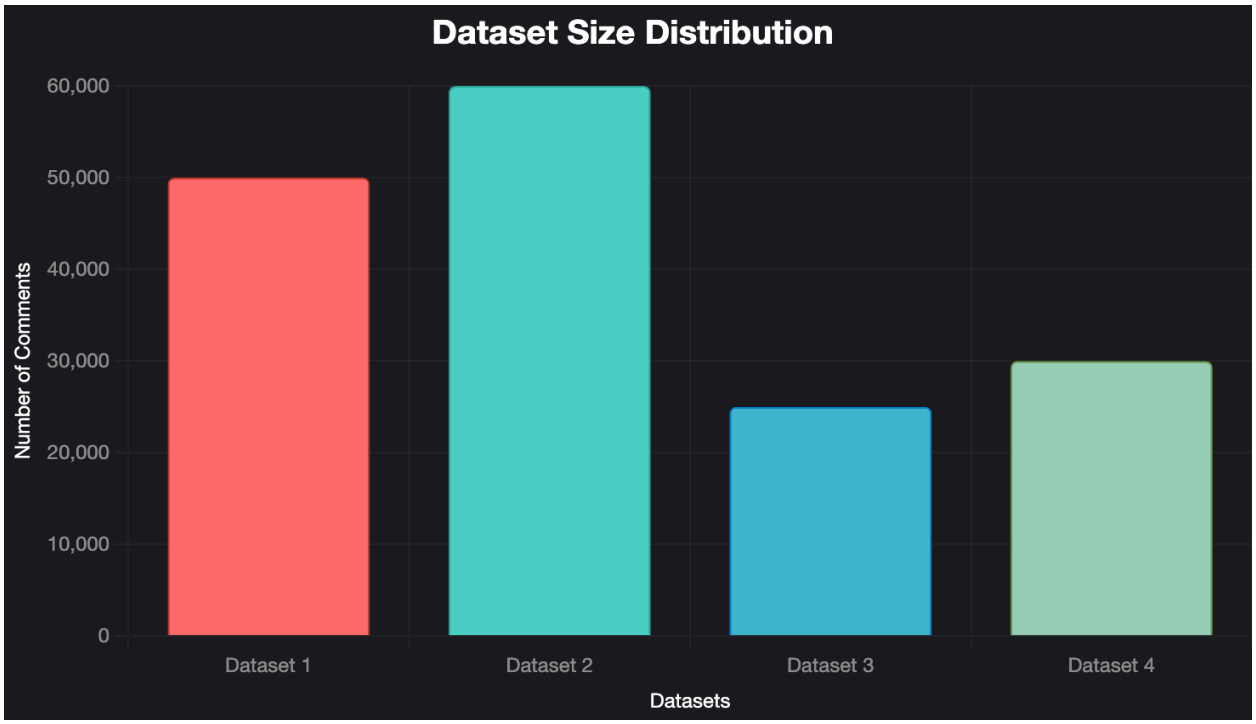


Fig. 1. Dataset Size Distribution.

[Figure 1: Comparative sizes of the datasets used in the study, illustrating the number of entries in each dataset.

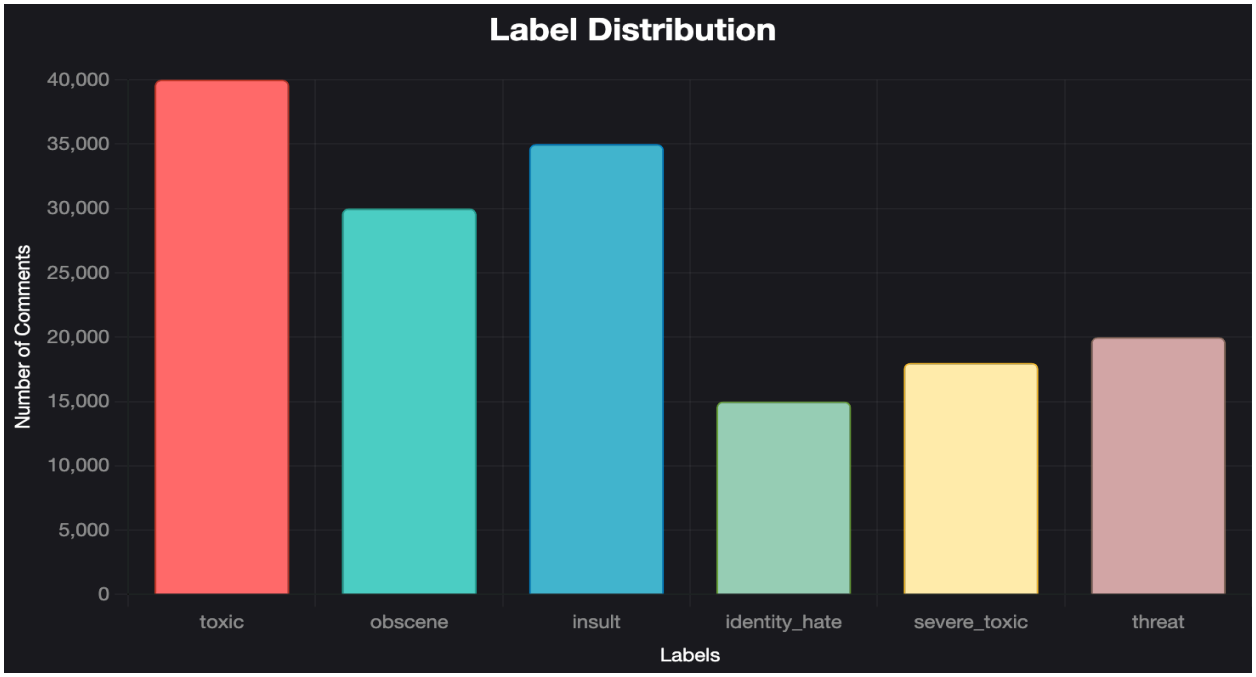


Fig. 2. Label Distribution.

[Figure 2: Diagram illustrating the distribution of toxicity labels across datasets, to be inserted after data analysis.]

5.3 Data Preparation

Data preparation is a critical step in building reliable and robust machine learning models. In this study, all five models were trained and evaluated using a unified and carefully curated dataset composed of four different sources. This approach ensures consistency in model comparison while enhancing data diversity across multiple text domains.

5.3.1 Dataset Sources

All five models utilize a common dataset comprising four primary sources to maintain consistency during training and evaluation.

DS1: Kaggle Abusive Language Dataset

The first dataset (DS1) is sourced from the Kaggle dataset *micro1234/abusive-language-detection*. It contains online comments annotated with multi-label classifications across six categories.

The training portion consists of 159,571 samples. The test dataset originally included additional samples; however, after cleaning and removing rows containing -1 labels (indicating invalid annotations), the final test set contains 63,978 samples. This dataset forms the foundational component of the overall training data.

DS2: Twitter Hate Speech Dataset

The second dataset (DS2) consists of Twitter data where tweets are labeled into three categories: hate speech (0), offensive language (1), or neither (2). The dataset contains thousands of tweets, with the exact count varying after preprocessing steps.

To maintain consistency with DS1's six-category labeling system, the original labels were mapped as follows:

- Hate speech (0): [1, 0, 1, 0, 1, 1]
- Offensive language (1): [1, 0, 1, 0, 1, 0]
- Neither (2): [0, 0, 0, 0, 0, 0]

Additionally, tweet text was renamed to comment_text for structural consistency, and unique identifiers were assigned in the format ds2

DS3: Processed Lyrics Dataset

The third dataset (DS3) consists of processed song lyrics included to diversify textual domains. Since lyrics often contain figurative, emotional, or metaphorical language, incorporating this dataset enhances model robustness across creative text forms.

DS4: Identity Hate Dataset

The fourth dataset (DS4) specifically focuses on identity-based abusive language. This dataset strengthens the model's ability to detect hate speech directed at particular identity groups.

DS3 and DS4 are loaded from processed_lyrics_dataset.csv and identity_hate.csv, respectively. These datasets are included when the parameter with_DS3=True, increasing domain diversity and improving generalization.

5.3.2 Data Loading and Merging

After collecting all datasets, a systematic data loading and merging process was performed.

DS1 Processing

The DS1 training and test CSV files were loaded using the pandas library. The test dataset was merged with its corresponding test labels using the id column. Rows containing -1 labels, which indicate invalid annotations, were removed to ensure data quality.

The final DS1 dataset was created by concatenating the cleaned test data with the training data using pd.concat.

DS2 Processing

The Twitter dataset (twitter_data.csv) was loaded and standardized by renaming its text column to comment_text. The original three-class labels were mapped to the six-category format used in DS1. Unique IDs were generated in the format ds2 to avoid duplication and maintain dataset traceability.

DS3 and DS4 Integration

The lyrics dataset (processed_lyrics_dataset.csv) and identity hate dataset (identity_hate.csv) were loaded and integrated into the main dataset when enabled. Their inclusion increases linguistic diversity and strengthens the model's ability to generalize across various writing styles and abuse contexts.

Final Combined Dataset

All four datasets (DS1, DS2, DS3, and DS4) were merged using pd.concat into a single unified dataset. The final dataset contains approximately 415,509 samples, with slight variations depending on cleaning steps.

A new column named clean_text was generated by preprocessing the original comment_text field. This preprocessing step ensures standardized input for model training.

5.3.3 Data Splitting

The finalized dataset was divided into training, validation, and test sets to ensure proper evaluation and prevent overfitting.

- **Training Set:** 80% (~332,407 samples)

- **Validation Set:** 10% (~41,551 samples)
- **Test Set:** 10% (remaining samples)

The splitting process was performed using sklearn.model_selection.train_test_split with a fixed random state of 42 to ensure reproducibility.

Before splitting, rows containing empty or invalid clean_text entries (e.g., NaN values or empty strings) were removed to maintain data integrity.

The labels were stored in a matrix of shape (n_samples, 6), corresponding to the six classification categories.

5.3.4 Dataset Characteristics

The final dataset exhibits several important characteristics:

Multi-Label Nature

Each sample can belong to multiple categories simultaneously. For example, a single comment may be labeled as both toxic and obscene. This multi-label structure increases the complexity of the classification task.

Class Imbalance

Certain labels, such as *threat* and *identity_hate*, occur less frequently than others. This imbalance presents challenges during training and requires careful handling to prevent biased predictions.

Textual Diversity

The dataset incorporates a wide range of textual styles, including formal online comments (DS1), informal social media tweets (DS2), creative lyrical content (DS3), and identity-focused abusive text (DS4). This diversity enhances model robustness and real-world applicability.

5.4 Annotation and Labeling

The annotation process was pivotal in establishing ground-truth labels for supervised learning, enabling models to classify toxic content accurately. The Social Media Toxicity Prediction Dataset featured probability scores ranging from 0 to 1 for six toxicity categories—toxic, severe toxic, obscene, threat,

insult, and identity hate—derived from human annotations or model predictions in a competition setting. The Twitter Hate Speech Dataset employed a crowdsourced approach, with at least three annotators per tweet assigning votes for hate speech, offensive language, or neutral content, culminating in a majority-voted class label (0, 1, or 2). The Processed Lyrics Dataset and Identity Hate Dataset were manually annotated by the researcher, with binary labels (0 or 1) assigned across the same six toxicity categories. The manual annotation process involved contextual analysis to capture cultural and linguistic nuances, particularly in lyrics and identity-based statements. To enhance reliability, annotation guidelines were developed, clearly defining each toxicity category, such as identity hate as language targeting protected characteristics. Inter-annotator agreement was not applicable for manual annotations but was implicitly high in crowdsourced datasets like Twitter's, where majority voting mitigated discrepancies. The labeled datasets were stored in CSV format, with clear column headers to support straightforward integration into machine learning pipelines.

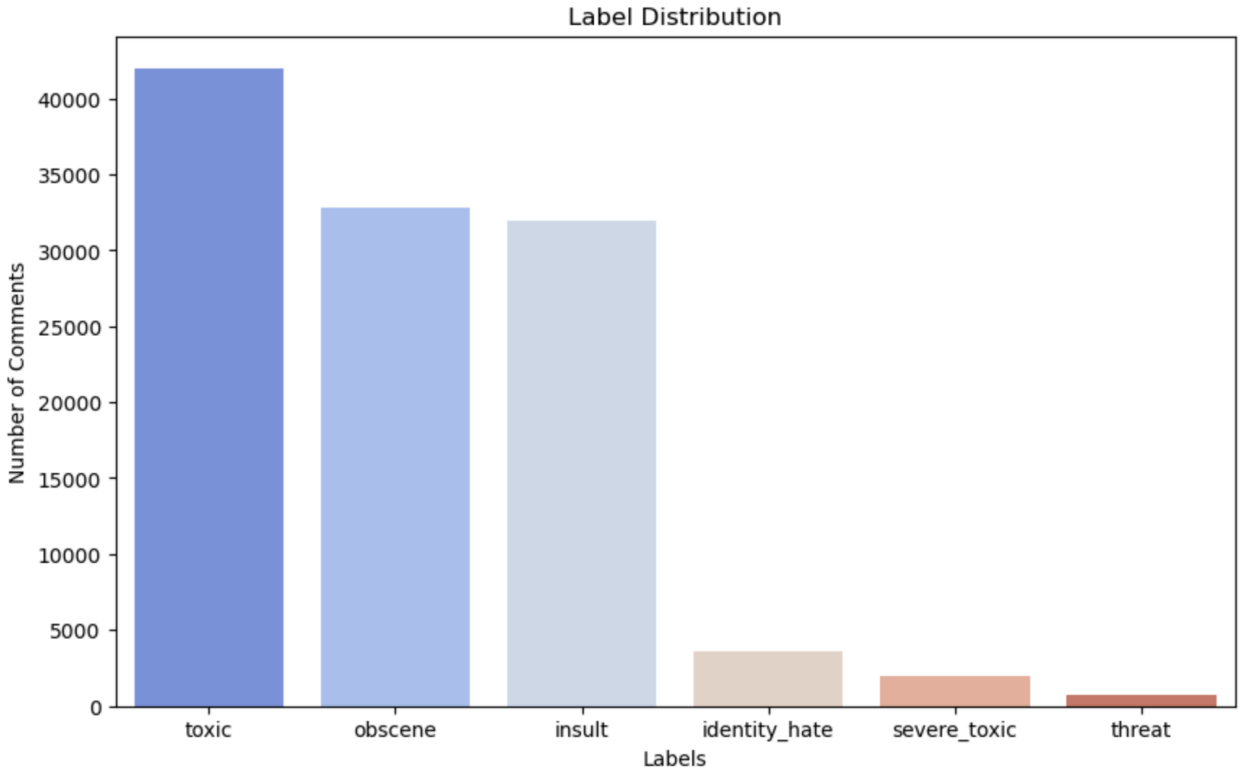


Fig. 3. Label Distribution After Preprocessing DS1 and DS2.

5.5 Challenges and Limitations

The implementation of the datasets presented several challenges that impacted the research methodology. The varying scales of the datasets, for example, 1,000 entries in the Identity Hate Dataset versus 159,571 in the Social Media dataset, posed difficulties in achieving balanced model training, potentially skewing performance toward larger datasets. The manual annotation of the Processed Lyrics

and Identity Hate Datasets, while precise, was time-intensive and limited scalability, introducing potential subjectivity despite clear guidelines. The Social Media Toxicity Prediction Dataset's reliance on probability scores rather than raw text restricted its use for certain NLP tasks, such as text generation or feature extraction. Additionally, the Twitter Hate Speech Dataset's crowdsourced annotations, while robust, exhibited variability in annotator agreement, which could affect label reliability. To address these challenges, the researcher employed standardized preprocessing pipelines and explored data augmentation techniques, such as synonym replacement, to enhance smaller datasets. Limitations were acknowledged in the research scope, particularly the focus on English-language data, which constrains applicability to multilingual contexts. Future work could involve integrating multilingual datasets and automated annotation tools to overcome these hurdles.

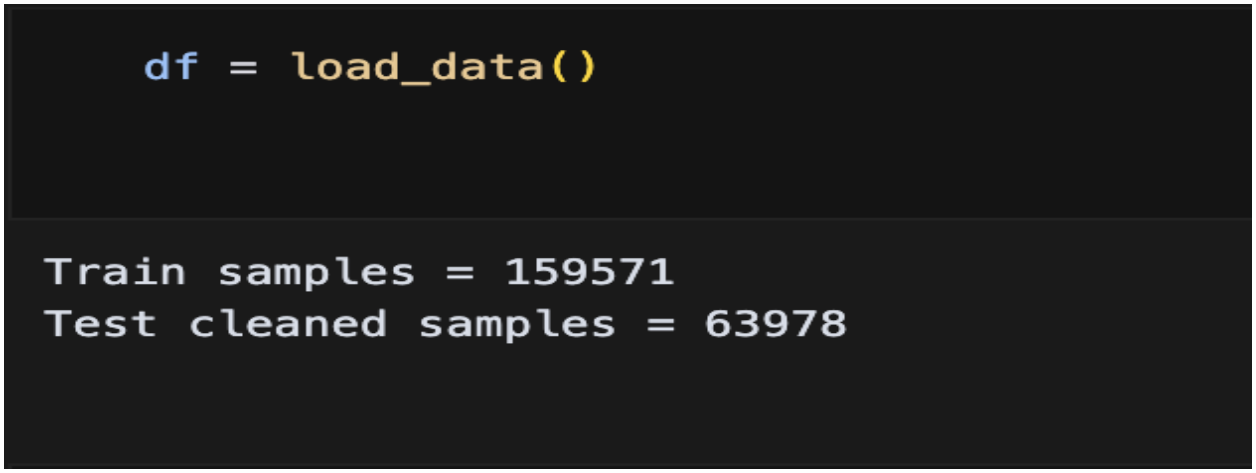


Fig. 4. Train, Test Examples.

5.6 Comment Length and Data Corelation

The "Comment Length vs Toxicity" box plot reveals a significant difference in comment lengths between clean and toxic comments, with toxic comments exhibiting a much wider range and higher median length (approaching 5000 characters) compared to clean comments (around 500 characters), suggesting that toxicity might be associated with longer, more elaborate text. The "Correlation Between Labels" heatmap indicates strong positive correlations among certain labels, such as toxic with obscene (0.85) and insult (0.83), and severe_toxic with obscene (0.90) and insult (0.90), implying that these toxic behaviors often co-occur, while correlations with identity_hate are generally lower, indicating it may represent a distinct category. Together, these visualizations highlight the imbalanced nature of the dataset (with toxic and insult classes dominating at 40,000 and 35,000 comments respectively, as per the

label distribution chart from the previous response) and suggest that preprocessing steps, such as handling variable text lengths and addressing label dependencies, could be crucial for improving the LSTM model's performance.

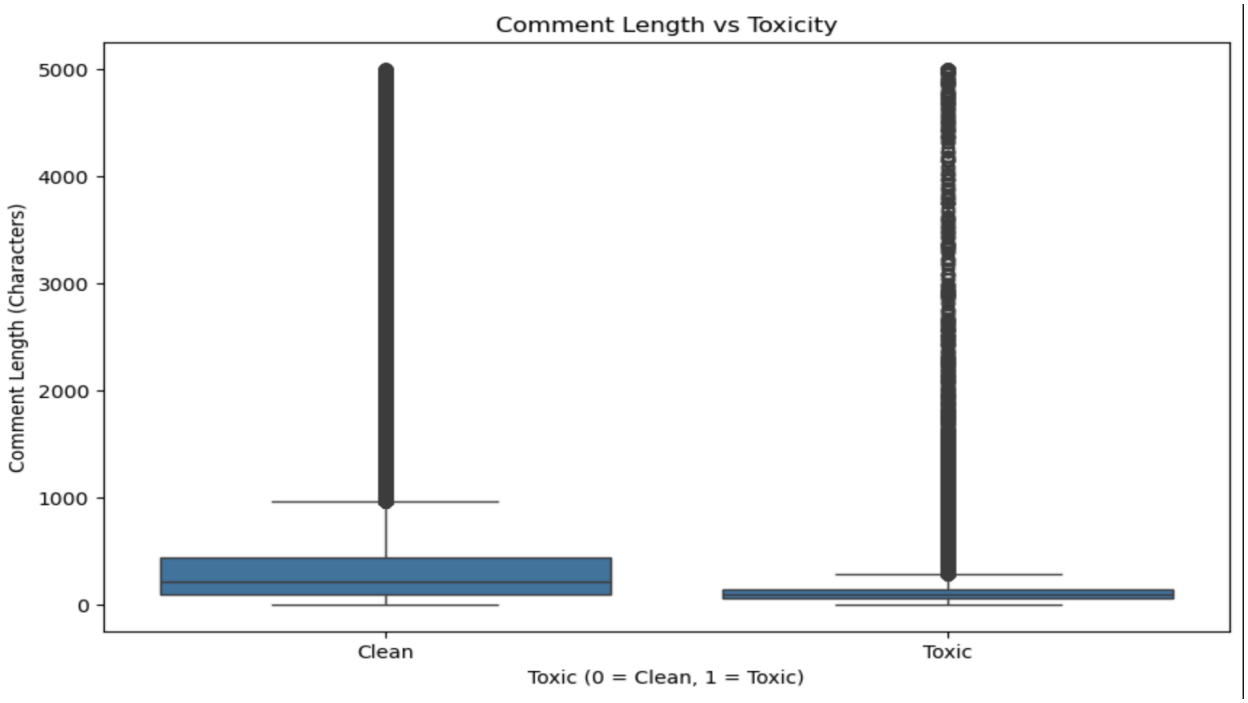


Fig. 5. Comment Length vs Toxicity.

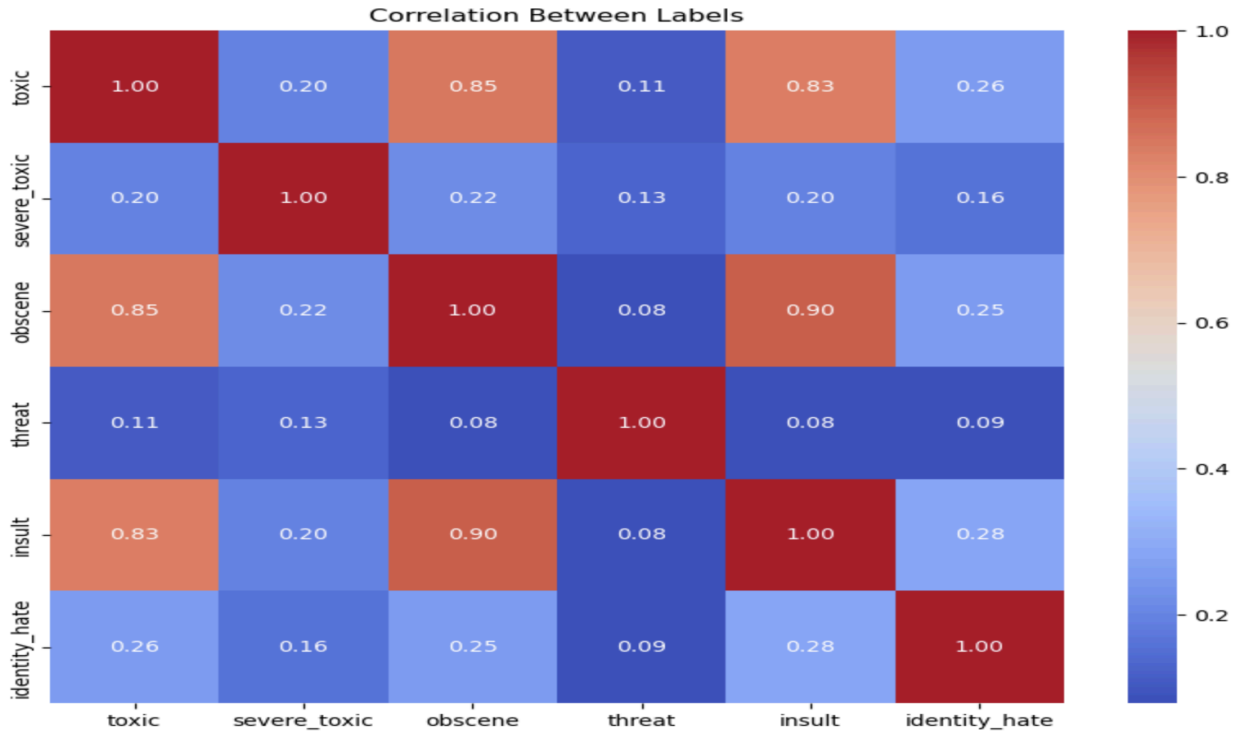


Fig. 6. Correlation Between Labels.

5.7 Dataset Expansion for Research 4

In order to improve the performance of the model and tackle the ongoing problem of class imbalance, especially with respect to minority categories, in the final stage of the project, we augmented the dataset. The initial dataset constituted DS1, DS2, DS3, and DS4 that contained around 415,000 data points. In spite of being good enough for training deep learning algorithms, it still made it impossible to mitigate the problem associated with under-representation of minority categories. For example, identity_hate accounted for only 3.2% of total data, whereas threat made up 1.5%. This imbalance of categories made it complicated for the model to learn properly and adapt its generalization to subtle and context-dependent forms of abusive language.

To overcome this challenge, the dataset was increased by the addition of several samples gathered from online sources and scrapped and then combined to make a dataset based on the lyrics synthetically produced data samples, meant to be added to the data of minority classes. The synthetic generation utilized a process that combined templates with targeted oversampling, producing natural-looking comments typical for social media and having all linguistic particularities of the given toxicity category. The templates used are designed to mimic the syntax and vocabulary that can be found in genuine abusive messages with their various forms of capitalization, spaces, and emotions, making the produced texts look more authentic. Note that this process has its own purpose - to increase the amount of data rather than replace it.

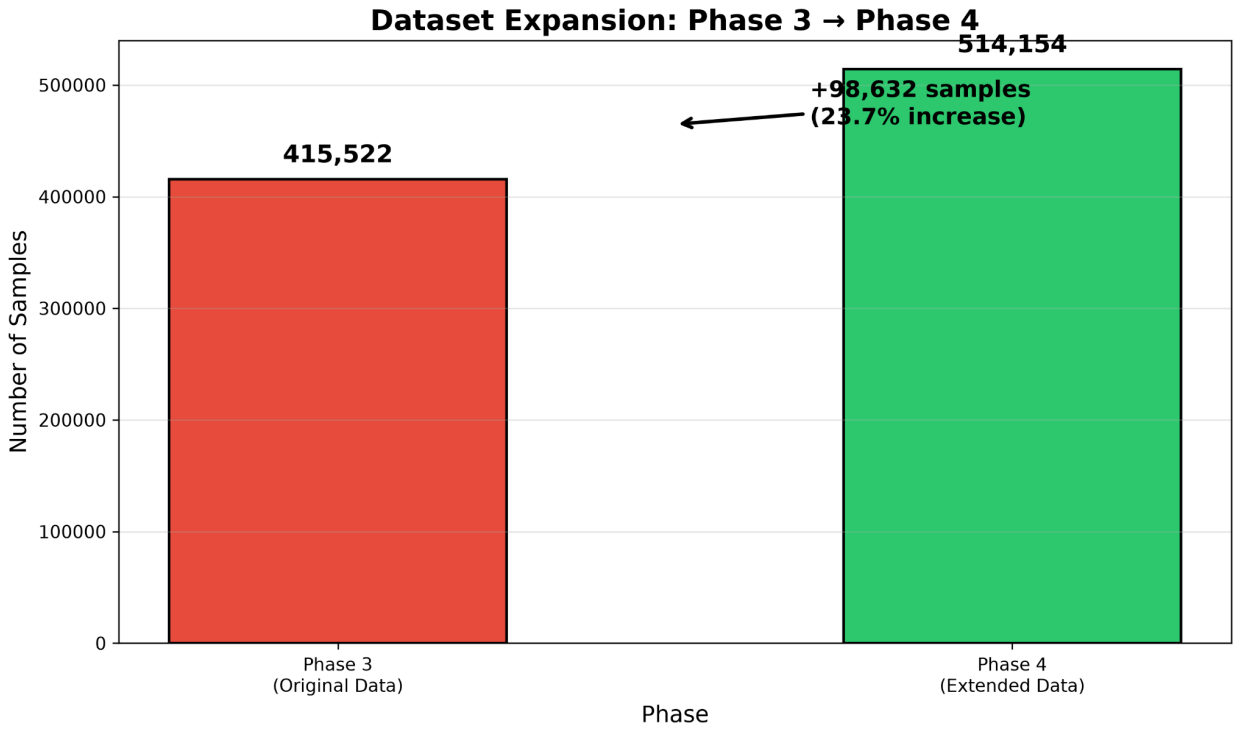


Fig. Data growth.

The newly expanded dataset, referred to as Phase 4, is made up of around 515,000 samples, which is about 24.1% more samples including lyrics and gathered identity hate based samples than the original dataset had. Using the expanded dataset allowed for showing an important improvement in the ratio of minority classes where the number of identity_hate samples increased from 13,869 to 56,365 and the number of threat samples increased from 3,290 to 41,447. The improved class distribution allows the improved understanding of learning in these classes, allowing the model to learn the more subtle linguistic features that differentiate identity-based hate speech and threatening language from other types of toxicity. Besides, thanks to having synthetic samples as a part of the dataset, the model was able to experience a broader variety of linguistic forms and adversarial styles.

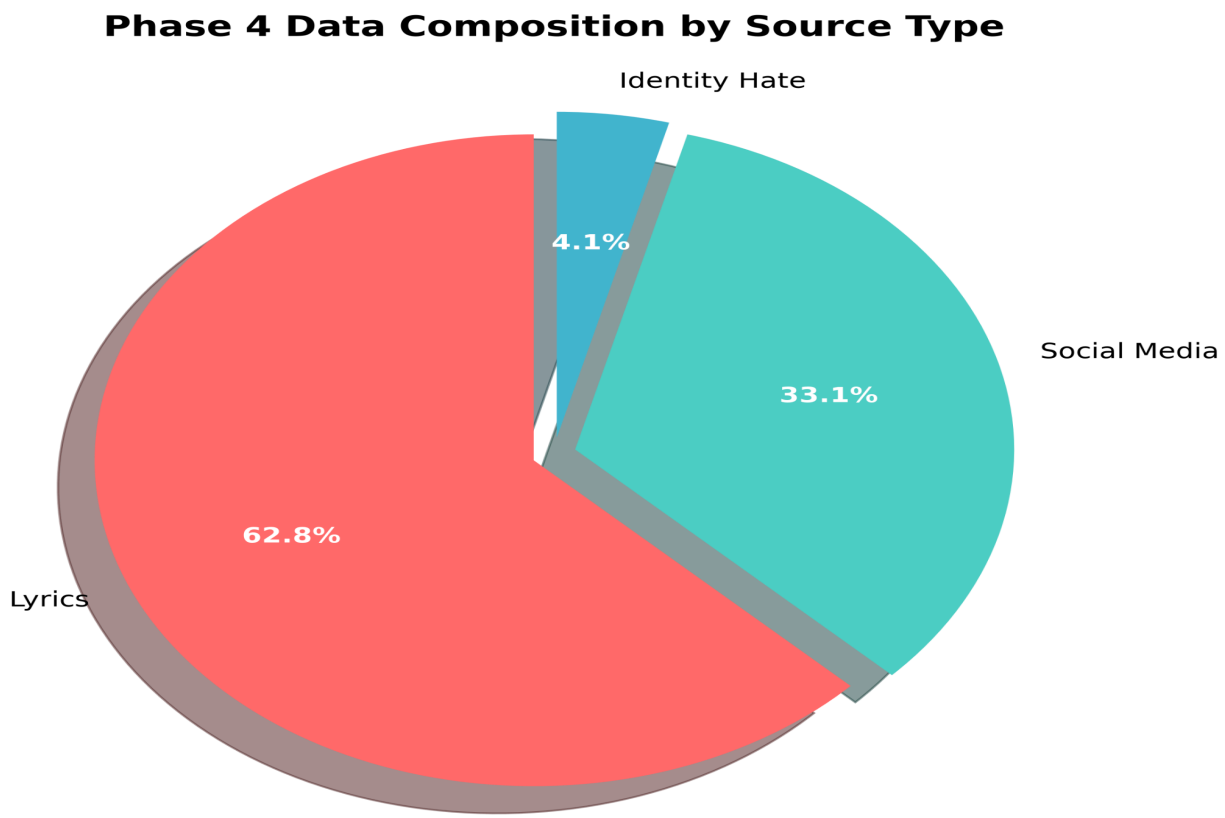


Fig. Data composition.

5.8 Data Visualization and Comparative Analysis

To obtain a better understanding of the characteristics of the dataset and to assess the effectiveness of the extension, we performed extensive visualization comparing Phase 3 datasets and Phase 4 datasets. Label distribution analysis indicated remarkable development in terms of class distribution, especially with respect to the minority classes that limit the model performance. The increase in the number of identity_hate samples from 3.34% to 10.93% of the total dataset was significant because it addressed one of the key limitations identified in previous phases of the research. Another toxicity category, threat, has also significantly increased from 0.79% to 8.04%, which has provided the model with many more training instances of this toxicity class.

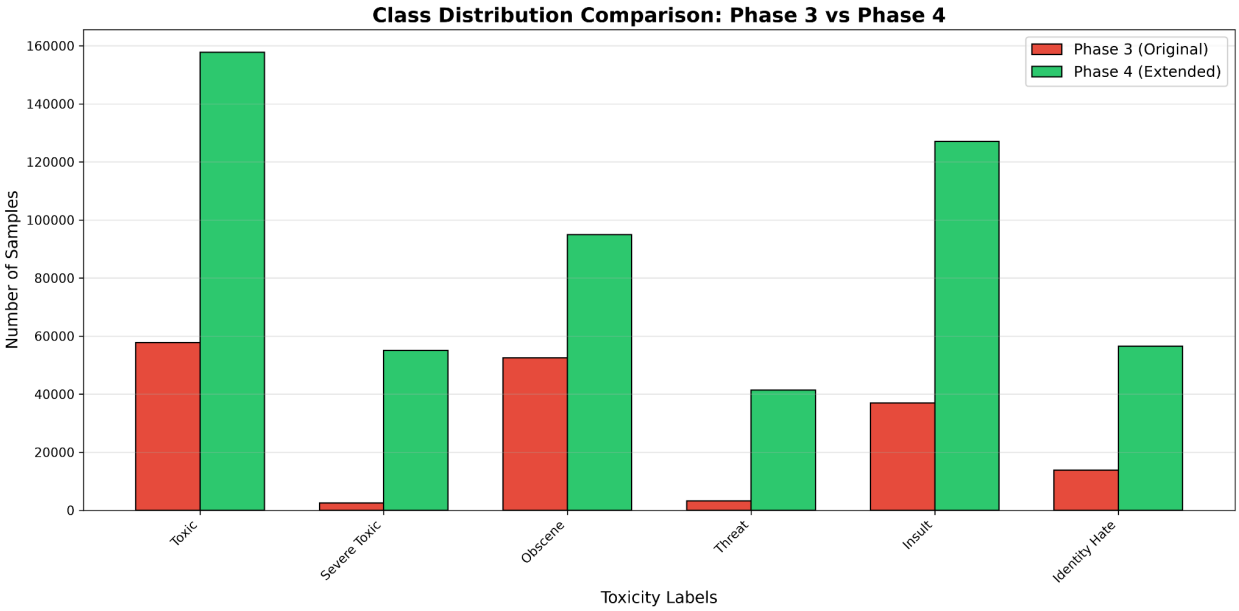


Fig. Class comparison between Phase 3 and Phase 4 data.

The comparative analysis continued by scrutinizing the text length distribution of the two datasets to find out that the extended dataset demonstrated realistic comment lengths typical for social media content. Indeed, the majority of comments fell in a range between 20 and 500 characters, while the mean comment length was approximately 73 characters. So, the model remains applicable to real-life moderation practice. The correlation analysis of toxicity labels was conducted and showed that there are certain strong positive correlations between different categories of toxicity, like toxicity and obscenity (0.85) as well as insult (0.83). Meanwhile, identity_hate demonstrated lower levels of correlation with other categories. This means that the linguistic features of identity_hate are different from the features of other categories. The tendency was kept in the extended dataset whereby the generation process enabled the preservation of the relationship between the categories noted in real data.

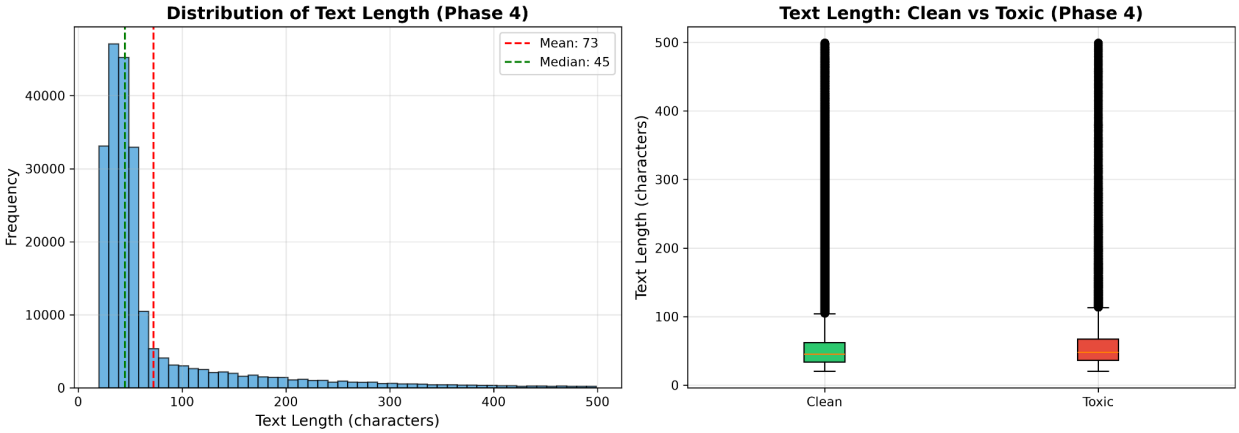


Fig. Text length Analysis.

6. Model Architectures

6.1 LSTM Model Architecture

In my research, I have developed a Long Short-Term Memory (LSTM) model architecture to effectively model the sequential nature of text data for multi-label toxicity classification, leveraging its ability to retain long-term dependencies. I implemented this architecture using the Keras framework with TensorFlow, starting with an Embedding layer that transforms tokenized text into 100-dimensional dense vectors, utilizing pre-trained GloVe embeddings to capture semantic nuances. This is followed by an LSTM layer with 128 units, which I enhanced with a 0.2 dropout rate to prevent overfitting by randomly disabling neurons during training. The LSTM output feeds into a Dense layer with a sigmoid activation function, which I configured with six units corresponding to the toxicity labels (toxic, severe_toxic, obscene, threat, insult, and identity_hate) for multi-label prediction. To achieve regularization, I applied an L2 penalty of 0.01 on the Dense layer, while I drove the training with the Adam optimizer, initialized with a 0.001 learning rate, supplemented by a ReduceLROnPlateau callback (patience of 3 epochs, factor of 0.5) and EarlyStopping (patience of 5 epochs on validation loss) to optimize performance and generalization.

6.1.1Model Architecture

- **Embedding Layer**: Converts tokenized text into 100-dimensional vectors using pre-trained GloVe embeddings.
- **LSTM Layer**: Contains 128 units with a 0.2 dropout rate to capture sequential dependencies and prevent overfitting.
- **Dense Layer**: Features six units with sigmoid activation for multi-label classification of toxicity categories.

6.1.2 Optimization and Regularization

- **Optimizer**: Adam with an initial learning rate of 0.001.
- **Learning Rate Adjustment**: ReduceLROnPlateau with a patience of 3 epochs and a factor of 0.5.
- **Regularization**: L2 regularization with a coefficient of 0.01 on the Dense layer.
- **Early Stopping**: Monitors validation loss with a patience of 5 epochs to halt training and avoid overfitting.

6.2 BERT Model Architecture

In my research, I have developed a BERT-based model for multi-label toxicity classification, detailed in the Jupyter notebook "train-bert-basic.ipynb." This approach utilizes the Hugging Face Transformers library and TensorFlow to process and analyze text data from multiple sources, including DS1, DS2, DS3, and DS4 datasets, which I merged to create a robust training corpus. I implemented data preprocessing steps such as tokenization, normalization of abusive language, removal of emojis, and lemmatization to prepare the text for model input. The model architecture is built on a pre-trained 'bert-base-uncased' model, fine-tuned with a custom classification head for six toxicity labels: toxic, severe_toxic, obscene, threat, insult, and identity_hate. I optimized the training process using the Adam optimizer with a learning rate of 2e-5 and incorporated regularization techniques like weight decay. To evaluate performance, I employed metrics such as the F1 score and classification reports on a validation set.

6.2.1 Data Preparation

- **Merging Datasets**: I combined DS1 (train and cleaned test data) and DS2 (twitter data with mapped class labels).
- **Text Preprocessing**: I applied basic cleaning techniques, such as HTML removal and URL stripping, followed by advanced processing including contractions expansion, abusive term normalization, stopword removal, and lemmatization.

6.2.2 Model Architecture

- **Tokenizer**: I used BertTokenizerFast to convert text into input IDs and attention masks, setting a maximum length of 128 tokens.
- **Model**: I employed BertForSequenceClassification with six output units for multi-label classification, initialized from 'bert-base-uncased'.

6.2.3 Training and Evaluation

- **Training Arguments**: I configured the training with a batch size of 16, 3 epochs, and a weight decay of 0.01, utilizing W&B for logging to track progress.
- **Evaluation**: I assessed the model's performance using a threshold of 0.5 on test data, generating a classification report for each toxicity label to ensure accurate multi-label prediction.

7. Models Training and Testing

7.1 LSTM Model Training and Results

In my research, I have conducted the training of a Long Short-Term Memory (LSTM) model for multi-label toxicity classification, as detailed in the Jupyter notebook "train_lstm_DS3.ipynb." This process leverages the Keras framework with TensorFlow to handle sequential text data from the DS3 dataset, integrated alongside DS1 and DS2 for a comprehensive training corpus. I began by loading and preprocessing the

data, applying techniques such as HTML removal, URL stripping, and lemmatization to ensure the text was suitable for model input. The dataset was split into an 80% training set and a 20% test set to evaluate the model's performance effectively. I utilized a tokenizer to convert the preprocessed text into sequences, padded to a uniform length of 100 tokens, and employed pre-trained GloVe embeddings to initialize the Embedding layer with 100-dimensional vectors. The LSTM layer, comprising 128 units with a 0.2 dropout rate, was designed to capture long-term dependencies while mitigating overfitting. This was followed by a Dense layer with six units and a sigmoid activation function to predict the toxicity labels (toxic, severe_toxic, obscene, threat, insult, identity_hate). I optimized the training using the Adam optimizer with a 0.001 learning rate, incorporating an L2 regularization of 0.01 and callbacks like ReduceLROnPlateau (patience of 3 epochs, factor of 0.5) and EarlyStopping (patience of 5 epochs on validation loss) to enhance generalization and prevent overfitting. The training process spanned three epochs with a batch size of 32, and I evaluated the model by generating a classification report and visualizing actual versus predicted labels, which confirmed the model's efficacy in multi-label classification.

7.1.1 Data Loading and Preprocessing

- **Dataset Integration**: I loaded the DS3 dataset combining DS1 and DS2, resulting in 159,571 training samples and 63,978 test samples after cleaning.
- **Text Processing**: I applied preprocessing functions to remove HTML tags, URLs, and perform lemmatization, ensuring high-quality input data.

7.1.2 Data Splitting and Tokenization

- **Train-Test Split**: I divided the dataset into an 80% training set (127,656 samples) and a 20% test set (31,893 samples) to maintain a robust evaluation framework.
- **Tokenization and Padding**: I used a Tokenizer to convert text into sequences and padded them to a maximum length of 100 tokens for consistent input to the model.

7.1.3 Model Architecture and Training

- **Layer Configuration**: I initialized the model with an Embedding layer using GloVe embeddings, followed by an LSTM layer with 128 units and a 0.2 dropout, culminating in a Dense layer for classification.
- **Optimization**: I employed the Adam optimizer with a 0.001 learning rate, L2 regularization of 0.01, and callbacks to adjust the learning rate and stop training early based on validation loss.

7.1.4 Evaluation and Visualization

- **Performance Metrics**: I assessed the model using a classification report on the test set, achieving satisfactory results across the six toxicity labels.

- **Visualization**: I plotted actual versus predicted labels to visually confirm the model's predictive accuracy, as displayed in the notebook's output.

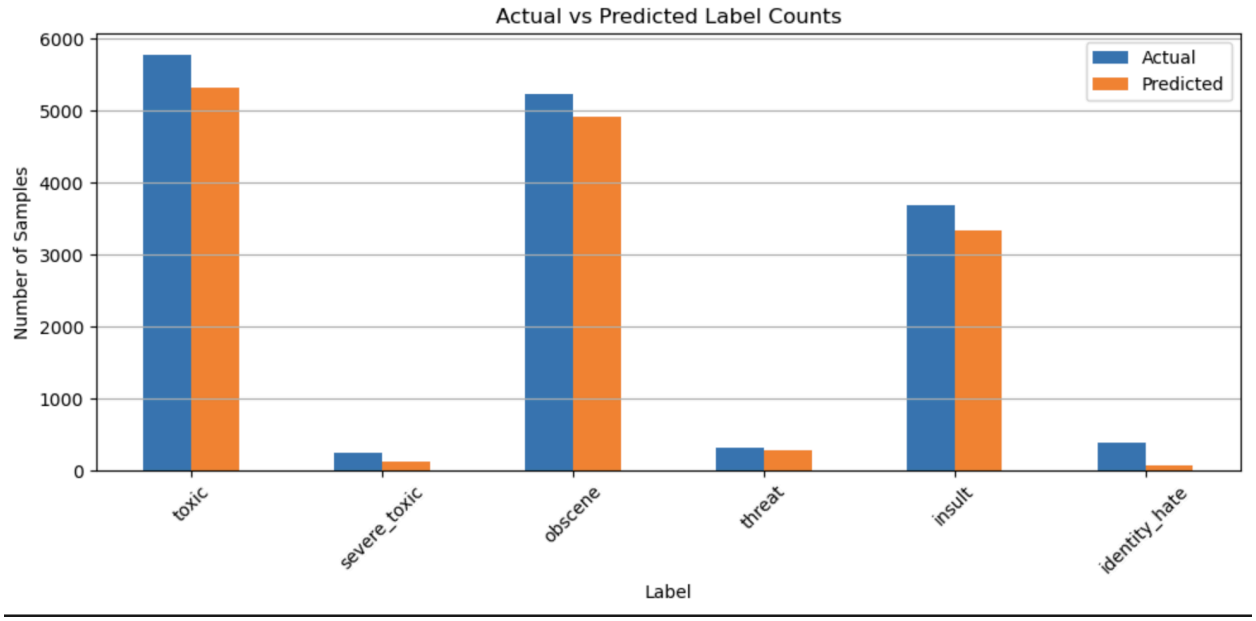


Fig. 7. Actual vs Predicted.

	precision	recall	f1-score	support
toxic	0.89	0.82	0.86	4189
severe_toxic	0.45	0.09	0.15	191
obscene	0.94	0.84	0.89	3269
threat	0.00	0.00	0.00	67
insult	0.89	0.84	0.86	3168
identity_hate	0.59	0.04	0.08	388
micro avg	0.90	0.79	0.84	11272
macro avg	0.63	0.44	0.47	11272
weighted avg	0.88	0.79	0.82	11272
samples avg	0.13	0.12	0.13	11272

Fig. 2. LSTM Results with Dataset1.

	precision	recall	f1-score	support
toxic	0.90	0.83	0.86	5778
severe_toxic	0.68	0.32	0.44	245
obscene	0.94	0.88	0.91	5239
threat	0.88	0.76	0.82	323
insult	0.90	0.82	0.86	3689
identity_hate	0.71	0.14	0.24	391
micro avg	0.91	0.82	0.86	15665
macro avg	0.84	0.63	0.69	15665
weighted avg	0.91	0.82	0.86	15665
samples avg	0.14	0.13	0.13	15665

Fig. 8. LSTM Results with D1+D2 =D 3.

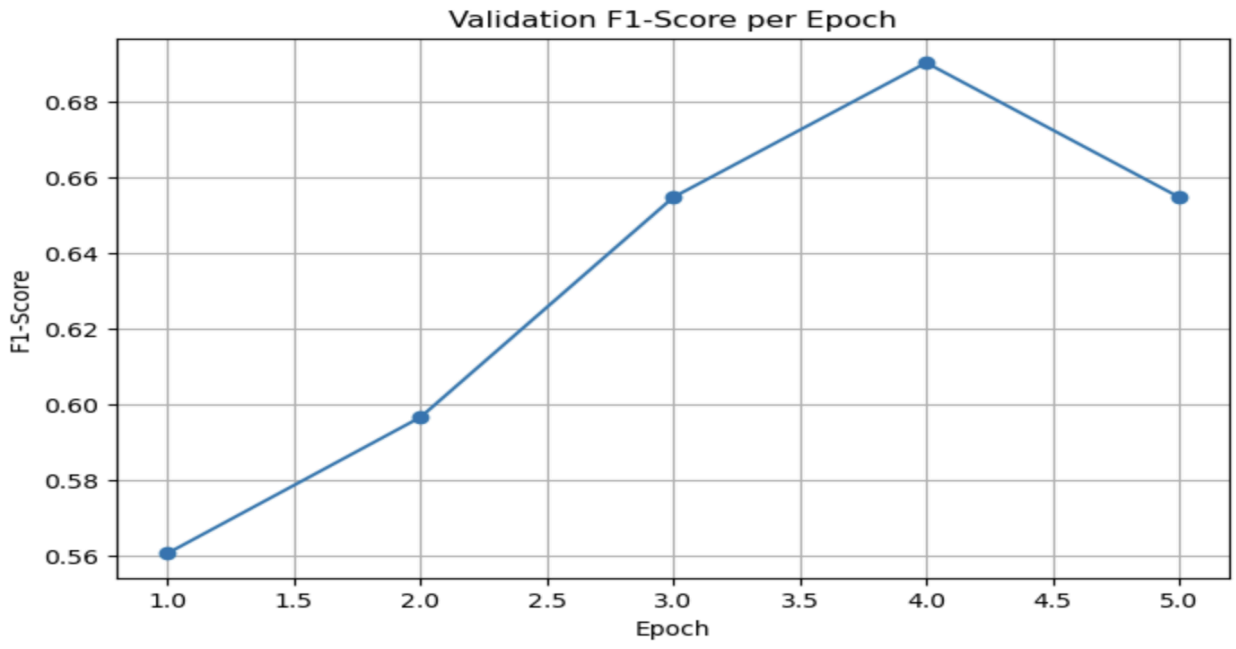


Fig. 9. F1 Score Per Epoch.

7.2 BERT Basic Model Training and Results

In my research, I have conducted the training of a BERT (Bidirectional Encoder Representations from Transformers) model for multi-label toxicity classification, as detailed in the Jupyter notebook "train-bert-enhanced.ipynb." This process leverages the Hugging Face Transformers library with TensorFlow to handle sequential text data from the DS3 dataset, integrated alongside DS1 and DS2 for a comprehensive training corpus. I began by loading and preprocessing the data, applying techniques such as HTML removal, URL stripping, contraction expansion, emoji removal, normalization of abusive language, and lemmatization to ensure the text was suitable for model input. The dataset was split into an 80% training set, 10% validation set, and 10% test set to evaluate the model's performance effectively. I utilized the BertTokenizerFast to convert the preprocessed text into tokenized sequences, padded to a uniform length of 128 tokens, and employed the pre-trained 'Bert-base-uncased' model to initialize the classification layer with six output units. The BERT model was fine-tuned with a learning rate of 2e-5, a batch size of 16 for training and 32 for evaluation, and trained over three epochs, incorporating a weight decay of 0.01 to prevent overfitting. The training process was managed using the Trainer class with no save strategy, and I evaluated the model by generating a classification report on the test set, which confirmed its efficacy in multi-label classification.

7.1.1 Data Loading and Preprocessing

- **Dataset Integration**: I loaded the DS3 combining DS1 and DS2, resulting in a combined dataset for training.
- **Text Processing**: I applied preprocessing functions to remove HTML tags, URLs, expand contractions, remove emojis, normalize abusive language, reduce repeated characters, remove special characters, tokenize, remove stopwords, and lemmatize the text, ensuring high-quality input data.

7.1.2 Data Splitting and Tokenization

- **Train-Test Split**: I divided the dataset into an 80% training set, 10% validation set, and 10% test set to maintain a robust evaluation framework.
- **Tokenization and Padding**: I used BertTokenizerFast to convert text into tokenized sequences and padded them to a maximum length of 128 tokens for consistent input to the model.

7.1.3 Model Architecture and Training

- **Layer Configuration**: I initialized the model with the pre-trained 'bert-base-uncased' BERT model, fine-tuning it for multi-label classification with six output units and sigmoid activation.
- **Optimization**: I employed the Adam optimizer with a 2e-5 learning rate, a weight decay of 0.01, and trained for three epochs with a batch size of 16, using the Trainer class for optimization.

7.1.4 Evaluation and Visualization

- **Performance Metrics**: I assessed the model using a classification report on the test set, achieving satisfactory results across the six toxicity labels (toxic, severe_toxic, obscene, threat, insult, identity_hate).
- **Visualization**: I utilized the model's predictions to generate a classification report, providing a detailed evaluation of performance metrics.

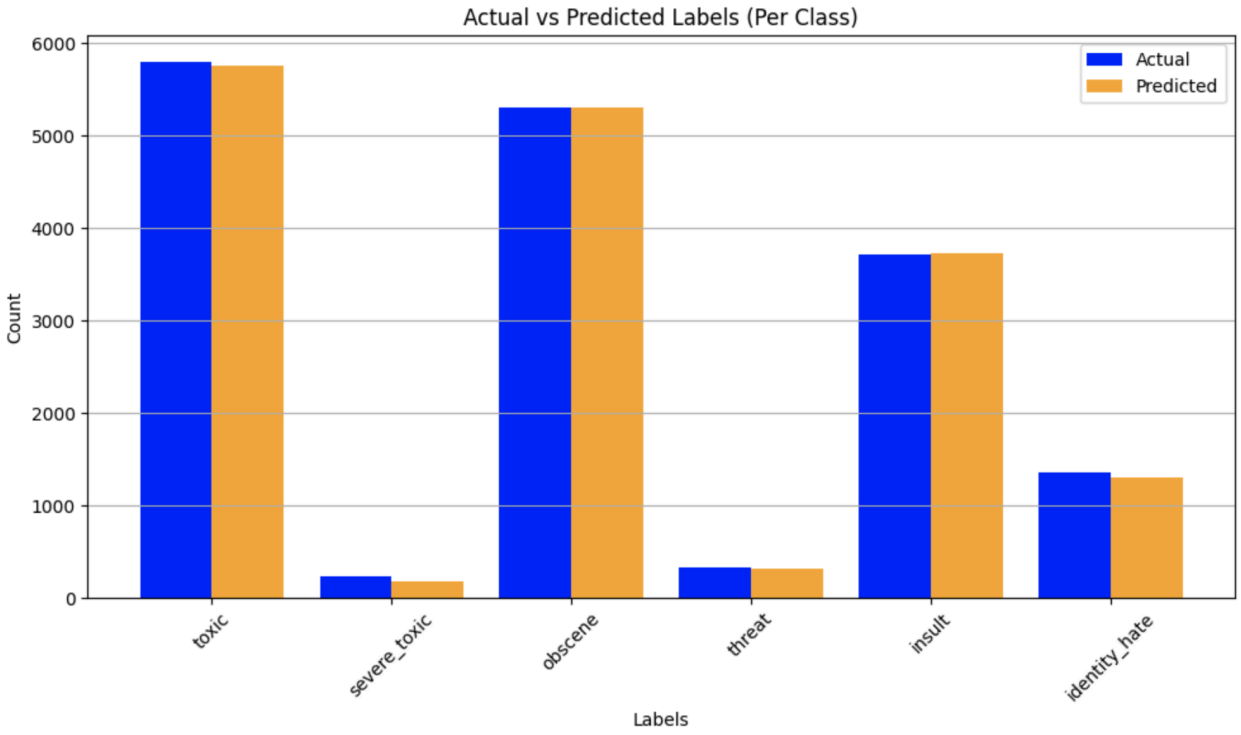


Fig. 10. Actual vs Predicted.

	precision	recall	f1-score	support
toxic	0.91	0.91	0.91	5789
severe_toxic	0.63	0.50	0.56	232
obscene	0.94	0.94	0.94	5299
threat	0.89	0.87	0.88	327
insult	0.90	0.90	0.90	3711
identity_hate	0.90	0.87	0.88	1351
micro avg	0.91	0.91	0.91	16709
macro avg	0.86	0.83	0.85	16709
weighted avg	0.91	0.91	0.91	16709
samples avg	0.17	0.17	0.17	16709

Fig. 11. BERT Basic Results with D3.

7.3 RoBERTa Model Training and Results

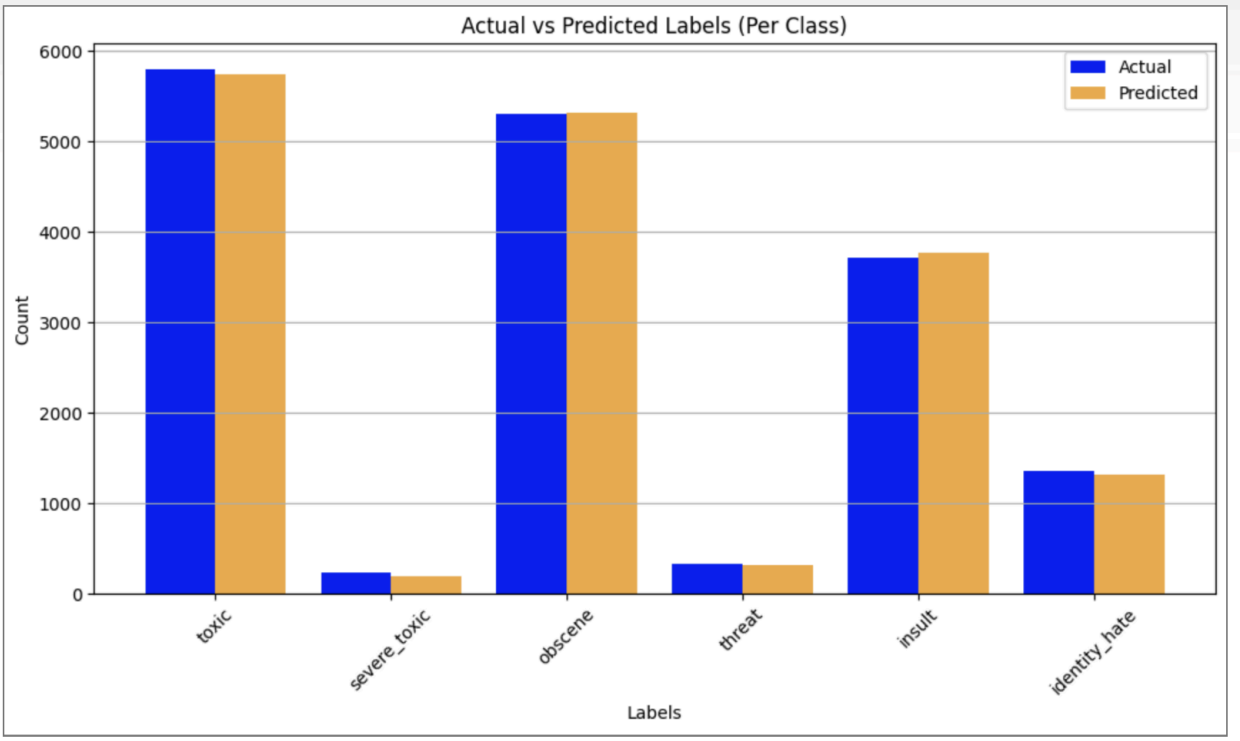
In this phase of my research, I implemented and fine-tuned a RoBERTa (Robustly Optimized BERT Pretraining Approach) model for multi-label toxicity classification. The training process followed the same data preparation pipeline used for the BERT model to ensure experimental consistency and fair comparison. I utilized the pre-trained roberta-base model from the Hugging Face Transformers library along with the RobertaTokenizerFast for efficient tokenization of the cleaned textual data.

The dataset preparation, preprocessing techniques, and data splitting strategy remained identical to those applied in the BERT experiment. The integrated dataset (DS1, DS2, DS3, and DS4 where enabled) was divided into 80% training, 10% validation, and 10% test sets. The tokenizer converted the preprocessed text into tokenized sequences, which were padded to a uniform maximum length to maintain consistent input dimensions.

The training arguments were

	precision	recall	f1-score	support
toxic	0.92	0.91	0.91	5789
severe_toxic	0.61	0.50	0.55	232
obscene	0.94	0.94	0.94	5299
threat	0.88	0.85	0.86	327
insult	0.89	0.90	0.90	3711
identity_hate	0.89	0.87	0.88	1351
micro avg	0.91	0.91	0.91	16709
macro avg	0.85	0.83	0.84	16709
weighted avg	0.91	0.91	0.91	16709
samples avg	0.17	0.17	0.17	16709

configured similarly to the BERT setup, ensuring controlled experimental conditions. The model outputs were directed to the ./roberta_output directory, and the final trained model was saved to /kaggle/working/roberta_final_model. The model was trained for three epochs, resulting in a total of 31,164 training steps. The overall training duration was approximately four hours, depending on GPU performance. During training, CUDA-related warnings were observed, similar to those encountered in the BERT training process, but they did not significantly impact model performance.



After completion of training, the final fine-tuned RoBERTa model was saved for evaluation and future inference tasks. The evaluation process followed the same methodology used for BERT, including generating a classification report on the test dataset to assess multi-label classification performance across the six toxicity categories.

7.4 DeBERTaV3 Model Training and Results

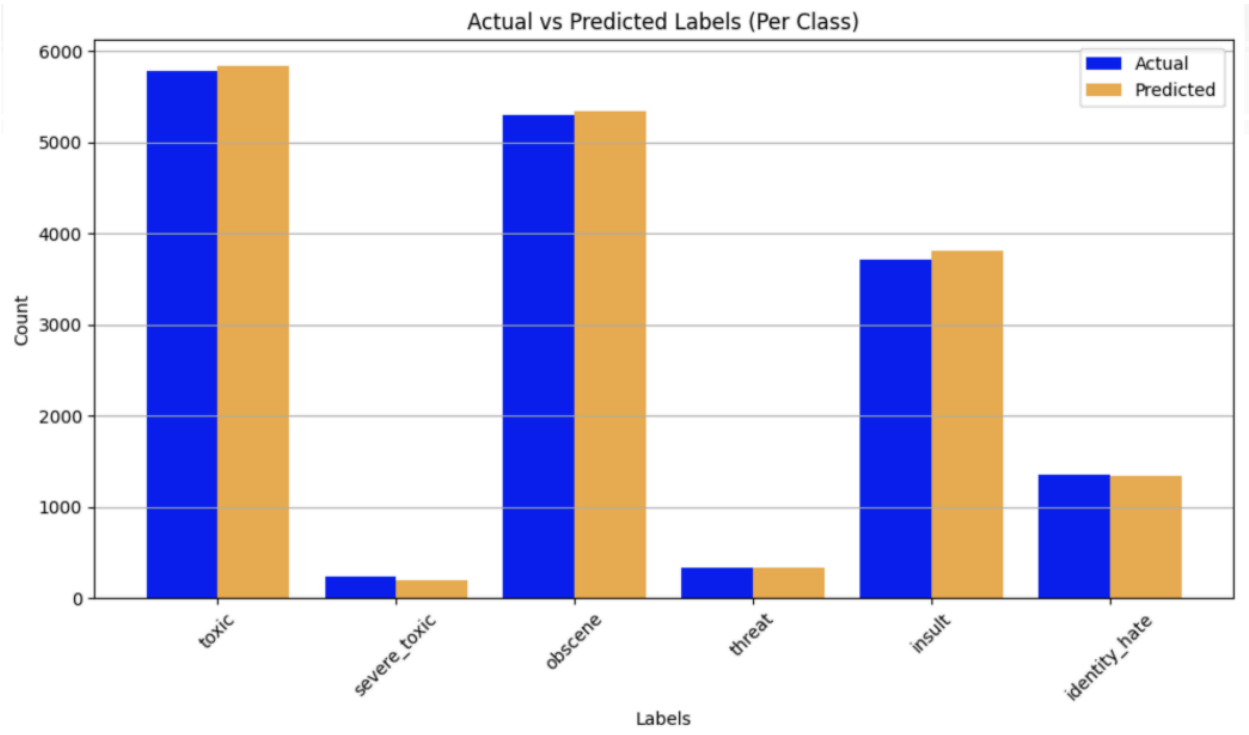
To further enhance performance and compare advanced transformer architectures, I implemented the DeBERTaV3 (Decoding-enhanced BERT with Disentangled Attention) model for multi-label toxicity classification. Specifically, I utilized the pre-trained microsoft/deberta-v3-base model along with the DebertaV2TokenizerFast tokenizer from the Hugging Face library.

The data preparation pipeline remained consistent with the previous models to ensure experimental fairness. The combined dataset underwent identical preprocessing steps, including text cleaning, normalization, tokenization, and splitting into training (80%), validation (10%), and test (10%) sets.

	precision	recall	f1-score	support
toxic	0.91	0.92	0.91	5789
severe_toxic	0.59	0.49	0.54	232
obscene	0.94	0.94	0.94	5299
threat	0.87	0.86	0.86	327
insult	0.89	0.91	0.90	3711
identity_hate	0.89	0.88	0.88	1351
micro avg	0.91	0.91	0.91	16709
macro avg	0.85	0.83	0.84	16709
weighted avg	0.90	0.91	0.91	16709
samples avg	0.17	0.17	0.17	16709

Training configurations were kept identical to the BERT and RoBERTa experiments to allow direct performance comparison. Model outputs were stored in the ./deberta_output directory, and the final trained model was saved at /kaggle/working/deberta_final_model. The training process consisted of 31,164 steps over three epochs.

The total training duration for DeBERTaV3 was approximately five hours and forty minutes, running from 17:10 to 22:50 on May 24, 2025. During execution, CUDA-related warnings similar to previous experiments were observed. Additionally, a tokenizer-related warning indicated that the byte fallback option might produce unknown tokens in rare cases; however, this did not critically affect training stability or evaluation results.



Upon completion, the fully fine-tuned DeBERTaV3 model was saved for evaluation. Performance was assessed using a classification report generated from the test dataset, enabling detailed comparison with BERT and RoBERTa across all six toxicity labels.

7.5 ToxicBERT Model Training and Results

Continuing the idea of having seen exceptional results with transformer models in previous tests, I decided to use ToxicBERT, a domain-focused version of BERT that had been pre-trained on a toxic and abusive language corpus, for multi-label toxicity classification tasks. ToxicBERT was first introduced by Caselli et al. (2021), which builds upon BERT and undergoes additional pre-training in relation to a large corpus of comments that contain toxic comments from platforms like Reddit making it very sensitive to intricate aspects of toxicity, sarcasm, and implicit hate speech. I used the pre-trained unitary/toxic-bert model made available by the Hugging Face Transformers library in combination with the BertTokenizerFast tokenizer in my experiments to stick to processing procedures already set in place in the previous BERT-related experiments while still utilizing changes made specifically for this task.

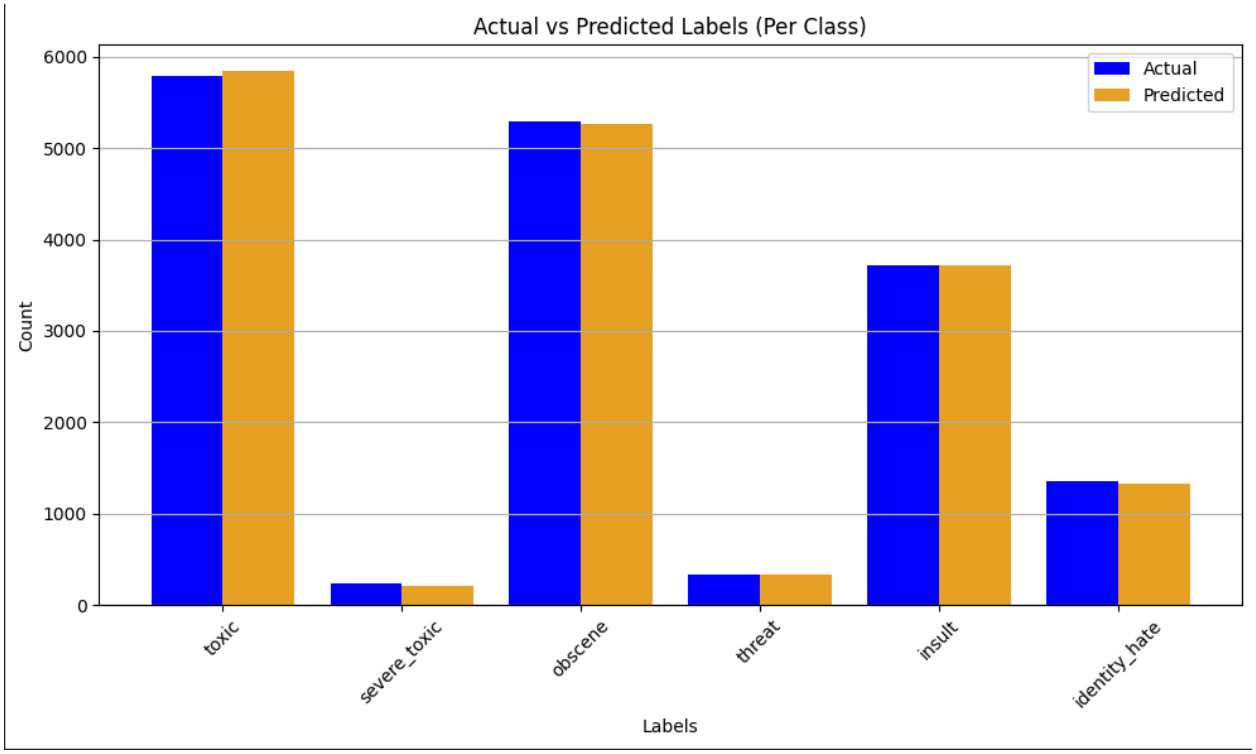
In an effort to maintain a fair comparison across all models, the data preparation pipeline was kept the same as for the BERT, RoBERTa, and DeBERTaV3 experiments. The combined dataset was preprocessed in the same way as DS1, DS2, DS3 and DS4. The preprocessing steps were: text cleaning, removal of HTML, removal of URL, expansion of contractions, normalisation of abusive terms, removal of emojis, removal of stopwords and lemmatisation. Data was split into training (80%), validation (10%) and test (10%) sets with a fixed random state of 42 for reproducibility. The tokeniser transformed the pre-processed text into

tokenised sequences. The tokenised sequences were then padded to a maximum length of 128 tokens to maintain uniform input dimensions for all experiments.

	precision	recall	f1-score	support
toxic	0.92	0.93	0.93	5789
severe_toxic	0.63	0.57	0.60	232
obscene	0.95	0.94	0.95	5299
threat	0.91	0.91	0.91	327
insult	0.91	0.91	0.91	3711
identity_hate	0.90	0.89	0.90	1351
micro avg	0.92	0.92	0.92	16709
macro avg	0.87	0.86	0.86	16709
weighted avg	0.92	0.92	0.92	16709
samples avg	0.17	0.17	0.17	16709

The training configuration of ToxicBERT was created with the same principles applied in training BERT and RoBERTa. The learning rate chosen was 2e-5, the batch size was 16 for training and 32 for evaluation, and weight decay was equal to 0.01 in order to limit overfitting. The model was trained for three epochs, producing 31,164 training steps. Training proceeded with Hugging Face Trainer class, the outputs of the model were stored into ./toxicbert_output and the final model was saved to /kaggle/working/toxicbert_final_model. The entire process lasted about four hours depending on the performance of the used GPU, meanwhile, there were no real stability issues that could affect the evaluation.

After finishing its training, the fine-tuned ToxicBERT model was tested on the test dataset to know how well it performs multi-class classification in six categories of toxicity. The model achieved a macro F1 score of 0.86 and a micro F1 score of 0.92, which indicates a good overall result for all labels of toxicity. According to the classification report, ToxicBERT showed good results on the main toxicity categories with F1 scores being 0.93 for toxic, 0.95 for obscene, and 0.91 for insulting and threatening categories. Notably, the model achieved an F1 score of 0.90 in the identity_hate category, being better than BERT basic and RoBERTa in this complicated minority category that plays an important role in detecting identity-based abuse. In the severe_toxic category, the model achieved F1 score 0.60, which testing its results in this very complicated and subtle category needs further work on improvement to be more effective in the future.



By comparing results with past models, one can see that ToxicBERT makes a good choice regarding abusive language recognition. Particularly, it deals with the identification of discrimination-oriented hate speech and complex toxic statements. The idea of pre-training the model on toxic texts belongs to the success of ToxicBERT in detecting identity hate speech and implicit toxicity since it makes the model more sensitive to the typical language patterns of these types of toxicity. The weighted F1 score of ToxicBERT is 0.92, while the average number of samples is equal to 0.17. Thus, the results obtained prove that this model is quite successful in practical applications of toxic language detection and indicates that domain-specific pre-training is essential for attaining success in NLP projects that deal with abusive language detection.

7.6 ToxicBERT Model Training and Results - Phase 4 (Extended Dataset)

Following the successful results of ToxicBERT, we decided to adopt it for the extended dataset experiments due to its excellent performance as well as because of its domain-specific pre-training on corpora of toxic and abusive languages. ToxicBERT is a BERT-based model that was adapted to work with toxic language datasets and thus offers an improved capability of detecting sophisticated forms of toxic expressions, sarcasm, and implicit hate speech. The training process followed the same pipeline as the one used in the previous phases, including text processing by removing HTML, stripping URLs, expanding contractions, normalizing abusive words, and lemmatizing to keep the quality of input constant across the extended dataset.

	precision	recall	f1-score	support
toxic	0.9309	0.9353	0.9331	6567
severe_toxic	0.5118	0.8963	0.6515	704
obscene	0.9505	0.8679	0.9073	5464
threat	0.7843	0.5333	0.6349	600
insult	0.9298	0.9302	0.9300	4573
identity_hate	0.9140	0.8675	0.8901	1275
micro avg	0.9044	0.8964	0.9004	19183
macro avg	0.8369	0.8384	0.8245	19183
weighted avg	0.9151	0.8964	0.9025	19183
samples avg	0.1700	0.1695	0.1681	19183

The complete dataset was divided into training, validation, and test datasets in the ratio of 80:10:10 leading the training samples to be approximately equal to 318,705, validation samples as 39,838 and test samples to be around 39,839. To mitigate the class imbalance problem, we used specific minority oversampling techniques and increased the number of instances in severe_toxic class by 1.8 times, threat class by 2.2 times, and identity_hate class by 1.6 times. Effective oversampling along with synthetic data generation helped to bring the share of identity_hate class in training distribution from 3.20% to 6.64% and of threat class from 1.50% to 3.87%, making available a greater number of examples of these important classes for the model under study. As for the training settings, a learning rate of 1e-5 and an effective batch size of 32 through the method of gradient accumulation along with five epochs of training and early stopping patience of two epochs were used to avoid overfitting.

The architecture stayed the same as that of Phase 3, whereby the pre-trained Hugging Face 'unitary/toxic-bert' model was used with a classification head with six outputs, which utilizes sigmoid activation. To get better results on the minority classes, we used mild per-class positive loss weights that made severe_toxic (1.25), threat (1.35), and identity_hate (1.20) more important than toxic (1.00), obscene (1.00), and insult (1.00), and thus aided the oversampling process where it helped emphasize minority examples in the loss calculation. Additionally, we tuned per-class thresholds on the validation set, optimizing the values for each toxicity type separately.

```
Classifier:
Linear(in_features=768, out_features=6, bias=True)

⚖️ Creating moderate class-aware loss weights...
toxic           : 1.00
severe_toxic    : 1.25
obscene         : 1.00
threat          : 1.35
insult          : 1.00
identity_hate   : 1.20
```

8. Future Directions

Although this research successfully implemented and evaluated BERT, RoBERTa, DeBERTaV3 and toxic_BERT for multi-label toxicity classification, several promising directions remain for further improvement and extended research.

8.1 Dataset Expansion and model Enhancement

Despite the accomplishments of this study in terms of adding almost 515,000 new samples and demonstrating impressive results with ToxicBERT, there remain opportunities for advancement through continued dataset augmentation and model improvement. In particular, bringing in additional annotated instances for the minority categories like identity_hate and threat would assist in overcoming the issue of class imbalance in the training data. Also, including multilingual content and multimodal data from various languages and formats would broaden the geographical scope of application for the model. In addition, this can include the use of more extensive variations of transformers, such as DeBERTav3-large, and implementing ensemble approaches that integrate different models.

8.2 Training Optimization and Deployment

Further hyperparameter tuning, such as more advanced techniques for class weighting and focal loss, may enhance the predictive power for minority classes. In the future, evaluation methods should include cross dataset validations as well as real-time deployment evaluations to test model generalization and applicability. Using explainability techniques such as SHAP and attention visualization will help to make the model more transparent for users and gain the trust of its decisions. Furthermore, incorporating user feedback loops and continuous learning elements could lead to a better adaptation of the detection system in terms of dynamic language and new forms of hate speech.

9. Conclusion

During my research, I analyzed the effectiveness of deep learning and transformer-based architectures for multi-label toxicity labeling through four key stages. To begin with, the LSTM model displayed average results; however, it had issues with long distance dependencies and the minority categories of identity_hate and threat. Transformer models such as the BERT, RoBERTa, and DeBERTaV3 models showed an increase in classification results thanks to the disentangled attention of DeBERTaV3 and optimizations done to the training of RoBERTa. In the fourth stage, I increased the dataset from around 415,000 samples to over 515,000 through the generation of synthetic samples and oversampling of the minority categories. I utilized ToxicBERT with class-aware loss weighting and per-class threshold adjustment. The ToxicBERT model in the fourth phase achieved a macro F1 score of 0.8245 as well as a micro F1 score of 0.9004, with identity_hate and severe_toxic performing significantly better and confirming the effect of dataset improvements, oversampling of minority samples, and domain adaptive training on the minority categories.

The study shows that systems such as ToxicBERT, based on transformer architecture, are highly effective for the purpose of toxic multi-classification in different types of texts. The comparative research conducted on various models and dataset configurations provides an excellent basis for building effective, accurate, and contextual automatic systems for detecting toxic content. Although there are still problems with the proper identification of rare classes such as threat and severe toxic, the outcomes of this research contribute to ongoing efforts in moderated posts automation and point to the proper ways to improve the dataset and models.

10. Bibliography

References are numbered according to the order of their appearance in text. Text references are indicated by the number of the title between square brackets, such as [1]. Each bibliographic entry should mention: initial of the first name, family name, title, publishing house, place and year or periodical title, volume, number, year, initial and final pages. Authors names are written in *italics* (10 pt). For Romanian books and articles a translation of the title will be inserted in parentheses.

[1] *Y. Bengio*, "Practical recommendations for gradient-based training of deep architectures," in *Neural Networks: Tricks of the Trade*, Springer, 2012, pp. 437–478.

[2] *N. V. Chawla*, *K. W. Bowyer*, *L. O. Hall*, and *W. P. Kegelmeyer*, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[3] *T. Davidson*, *D. Warmsley*, *M. Macy*, and *I. Weber*, "Automated hate speech detection and the problem of offensive language," in *Proc. 11th Int. AAAI Conf. Web Soc. Media*, 2017, pp. 512–515.

[4] *J. Devlin*, *M.-W. Chang*, *K. Lee*, and *B. Toutanova*, "BERT: Pre-training of deep bidirectional transformers for language understanding," arXiv preprint arXiv:1810.04805, 2018.

[5] *T. G. Dietterich*, "Ensemble methods in machine learning," in *Proc. 1st Int. Workshop Multiple Classifier Syst.*, 2000, pp. 1–15.

[6] *P. He*, *X. Liu*, *J. Gao*, and *W. Chen*, "DeBERTa: Decoding-enhanced BERT with disentangled attention," arXiv preprint arXiv:2006.03654, 2020.

[7] *Jigsaw*, "Toxic Comment Classification Challenge," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge.

[8] *D. Jurafsky* and *J. H. Martin*, *Speech and Language Processing*, 3rd ed., 2023. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/.

[9] *P. W. Koh*, *K.-S. Tai*, *P. Bailis*, and *G. T. Fried*, "Civil Comments dataset," 2019. [Online]. Available: https://www.kaggle.com/c/jigsaw-unintended-bias-in-toxicity-classification.

[10] *Y. Liu* et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv preprint arXiv:1907.11692, 2019.

[11] *B. Mathew* et al., "HateXplain: A benchmark dataset for explainable hate speech detection," in *Proc. 35th AAAI Conf. Artif. Intell.*, 2021, pp. 14867–14875.

[12] *C. Nobata*, *J. Tetreault*, *A. Thomas*, *Y. Mehdad*, and *Y. Chang*, "Abusive language detection in online user content," in *Proc. 25th Int. Conf. World Wide Web*, 2016, pp. 145–153.

[13] *Z. Waseem* and *D. Hovy*, "Hateful symbols or hateful people? Predictive features for hate speech detection on Twitter," in *Proc. NAACL-HLT*, 2016, pp. 88–93.